Final Year Project

An Application for Designing Optimal Switching Networks

Turlough Kelly

Student ID: 20343033

A thesis submitted in part fulfilment of the degree of

BSc. (Hons.) in Computer Science

Supervisor: Professor John Murphy



UCD School of Computer Science

University College Dublin

April 26, 2024

Table of Contents

1 Project Specification 4

1.1 Core 4

1.2 Advanced 5

2 Introduction 6

3 Related Work and Ideas 8

3.1 Existing Solutions and Applications 8

3.2 Analysis of Solutions 11

3.3 Justification of Application 12

3.4 Related Research Papers, Articles, and Works 13

4 Project Work-plan 18

4.1 Period 1: 11/09/23 - 31/12/23 18

4.2 Period 2: 22/01/24 - 26/04/24 20

5 System Design 22

5.1 Assembling the Required Formulae 22

5.2 Understanding the Topology 30

5.3 Choosing our Language and Framework 31

5.4 Code Implementation 33

6 Results 43

7 Future Work 47

8 Summary and Conclusions 48

This application is hosted via GitLab.

The link to this app can be found here.

Should you have any issues in accessing the above link, please contact me at turlough.kelly@ucdconnect.ie.

Abstract

Clos switches are a popular type of switch used in telecommunications and data centres [1] worldwide, which has recently found applications within neural networks[2] and cloud computing[3]. First formalised by Charles Clos in 1952, many networks today utilise the Clos switch alongside the Slepian switch (more commonly known as a Beneš switch[4]) to achieve their goals, which more recently has included machine learning[5].

The purpose of this project is to design optimal networks based on the user's specifications, such as the number of stages in the network, the types of switches used, and the overall size of the network. Using this information, the application will produce an interactive diagram showcasing the built network, alongside the minute details calculated for that specific network. It aims to provide a streamlined process for network engineers and researchers alike to aid them in their design of Clos and Slepian networks for practical implementation or for research purposes.

Chapter 1: Project Specification

"There are many types of switches used in Telecommunications and within Data Centres and two of the more common types are the Clos and Slepian switch. This project will attempt to develop an app that will be able to take as input the size of the switch and output the optimal number of stages and configuration of the switch, along with the graphical representation of it." - *original project specification*

The aim of this project is to develop a tool to aid in the designing of an optimal Clos or Slepian network based on a specified input, and if desired the number of stages and switch size (for Slepian networks). It will provide a simple topology of the created network, showcasing the required number of stages, dimensions of each switch, and all other relevant information about the created network. The user will have complete control over the creation of the network, allowing it to be tailored specifically to their needs.

The main goals that will be addressed through this project, alongside several auxiliary goals specifically related to the implementation of the application, are as follows:

1.1 Core

1. Compile all the formulae required to calculate the specific details of a Clos or Slepian topology, and organise them by switch type.
2. Implement the above functions in Java and ensure they function as intended.
3. Create an area within the application which will allow the user to view the equations for each switch separately, and provide tutorials on how to use them.
4. Create an interactable topology that showcases the created network from the user's input.
5. For larger networks, allow the user to view internal networks that form part of the whole network, detailing information specific to that internal network.
6. Allow the creation of sub-optimal topology should the user wish to do so.

1.2 Advanced

1. Are the given calculations optimal? Reverse engineer the formulae for each switch and investigate methods of expediting program execution time.
2. Should users specify sub-optimal networks, showcase areas on the diagram in which the network could be improved (I.E. increase/decrease the number of stages or change the variables derived from the input) and recommend changes.

Chapter 2: **Introduction**

In today's digital age, networks are the one of the many pillars that technology stands on, allowing it to power many everyday applications such as cloud computing[3], telecommunications[6], distributing computing[7] and high performance processing[2]. Furthermore, since the inception of the Internet as we know it today, networks have been critical for modern science and research, allowing communication between researchers from around the globe at a moment's notice[8]. For this reason, it is paramount that these networks are created optimally to ensure the best performance possible, in order to allow users of these services to enjoy a seamless experience. In particular, we will be focusing on a network type developed in the 20th century: the Clos network, as well as a sub-type of the Clos network, the Slepian network.

First invented by Edson Erwin in 1938[9] and officially formalised by Charles Clos in 1953, the Clos network is a strictly non-blocking, high capacity, multi-stage switching network[1] used in switching fabrics across large-scale data centres [3]. Initially, it provided a highly effective method of routing calls on mechanical telephone systems. While it was overtaken by further advancements in switching technology in the years following its formalisation, it later became the basis of switch operations for ethernet data communications in the 1990s, and serves as the basis of many large-scale, interconnected fabrics to this day. For this reason, Clos networks are vital for various network-based operations around the world.

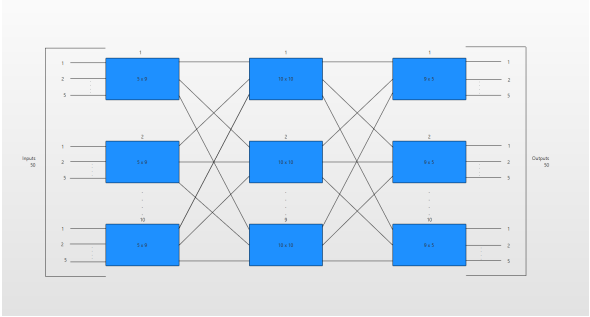


Figure 2.1: A simple representation of a Clos Network

A typical topology of a Clos network is represented as having 3 sections known as "stages", each containing parts known as "switches". The topology details these stages and the number of switches per stage, in addition to other information such as the dimensions of each switch, the number of inputs and outputs, and how each switch is connected to one another. The 3 stages include the input stage, the middle stage, and the output stage. It is important to note that for networks with larger stage counts (such as 5 stages), the middle stage will contain nested networks as its middle stage, meaning the middle of the 3 stages is a separate, yet connected, topology itself [1]. For example, in the case of a 5-stage network, while it may be represented in a similar fashion to a 3-stage network, the middle stage is in fact its own 3-stage topology, shown simply as a single stage.

For any k-stage topology, the middle stage will always be a $(k-2)$-stage topology represented as a middle stage. This extends further should the middle stage be greater than 3. For example, in the case of a 7-stage Clos network, the middle stage would be a 5-stage network, which would also contain a 3-stage network as its middle stage. This trait is also shared by its sister network, the Slepian network.

The Slepian network (more commonly referred to as a Beneš Network) is a sub-type of the Clos network, first proven feasible by David Slepian in 1955 and later refined by Václav Edvard Beneš by 1962[4]. Sharing many of the same use cases as a Clos Network, the Slepian Network is a rearrangeable, non-blocking network used in telecommunications. These networks have also found use in multi-processor systems and parallel computers, given their innate non-blocking characteristics and rearranging potential: a key difference from the Clos network. Slepian networks are more commonly used on smaller-scale networks[10], as opposed to larger networks. For this reason, the Clos and Slepian Networks have very different use cases, despite their innate similarities.

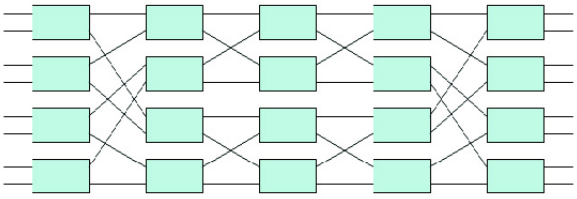


Figure 2.2: Slepian Network overview (credit: Kiarash Aghakhani, Research Gate[10])

A Slepian network architecture may be chosen over a Clos network due to its known ability to efficiently route permutations. This is achieved through its ability to dynamically reroute existing calls to different pathways through the switches present at each stage. In applications that may require this kind of dynamic rearrangement, the Slepian network may be chosen over a Clos network[4].

When comparing the topology of a Slepian network to that of a Clos network, one can note that there exists many similarities between them. For example, a Slepian network can be comprised of a 3-stage, 5-stage or 7-stage topology, almost identically to a Clos network. However, it is quite common for a Slepian network to reach much larger stages at a far more accelerated pace. This is most commonly seen in cases where the number of inputs per switch is a set number, such as a 2x2 switch, meaning each switch has 2 inputs and 2 outputs[10]. Both the Clos and Slepian networks have N inputs and outputs. However, the number of switches in each stage, and the dimensions of these switches are different depending on the network. In general, the computations for a Slepian network, while similar, are different to the computations related to a Clos network. A complete breakdown of the intricacies of the Clos and Slepian networks on a more technical level can be found in Chapter 5: Outline of Approach, Section 5.1.

Few tools exist that implement the topology of these networks for both implementation and research purposes. Indeed, many applications that support Clos and Slepian networks today focus on how they would interact with an existing network environment, and the costs involved with such an implementation. As important as these tools are, I wish to focus more on the theoretical aspect of Clos and Slepian networks and how they are constructed from a bottom-up perspective. Despite this theoretical focus, it can also serve as a simplistic starting point should the user wish to implement them into new or existing networks.

Chapter 3: Related Work and Ideas

There are various applications that are available to aid users in designing diagrams and topologies, catering to both general-purpose and network-specific needs. Consequently, numerous benchmarks exist against which new applications aiming for similar goals can be evaluated. Despite sharing the same end goal, the solutions offered by these applications can vary widely. Thus, there may be instances where multiple applications partially cover, but do not entirely fulfil, a specific niche. Analysing these concepts enables us to identify potential shortcomings in the application I intend to develop and enhance its advantages. Rather than replacing existing applications, it will serve as an additional solution alongside them.

3.1 Existing Solutions and Applications

Applications for designing networks for various purposes are not new. This field has seen significant investment, especially given the rapid evolution of technology. Existing applications encompass a wide range, including in-depth simulators and emulators, simple design tools, vendor-specific software, and applications capable of implementing networks and detailing traffic information, among others. It is prudent to explore these solutions to understand what has been accomplished and identify areas for potential improvement.

3.1.1 Network Design Software and Emulators

Network Design Software (NDS) primarily encompasses applications or tools designed to facilitate the modelling, planning, visualization, and sometimes implementation of computer networks. These tools aim to assist in building and optimizing various types of computer networks, including Clos networks. Generally, NDS provides a detailed and customisable view of the implemented network, tailored to specific scenarios. Additionally, many NDS incorporate emulators, which enable a host computer system to mimic the behaviour of another system (the guest) through software, hardware, or both. Emulators play a vital role in computing, ranging from logic simulators for digital circuit creation to functional emulators for simulating programs written in assembly or compiler languages. In networking, emulators create digital sandboxes to simulate network behaviour in various scenarios.

Examples of these NDS include, but are not limited to, the following:

Cisco Packet Tracer

Cisco Packet Tracer is a cross-platform visualisation tool developed by Cisco Systems, allowing users to design and simulate modern-day computer networks through routers, switches, servers and more, including Clos switches. It boasts many robust features to achieve this, such as support for various networking protocols and technologies, information on how the network integrates with hardware, simulation of traffic through the network, as well as diagramming and visualisation.

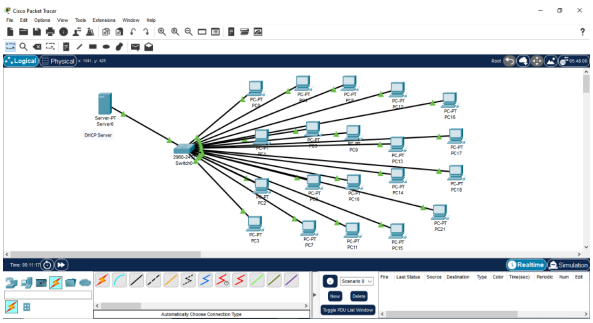


Figure 3.1: Sample Network built in Packet Tracer (credit: Geeks4Geeks)

In general, Packet Tracer is used in an educational capacity, and thus is easy to set-up and use, featuring a drag-and-drop user interface. The software provides a practical overview of the network, and how the different parts of said network interact with each other and with hardware.

Graphical Network Simulator-3 (GNS3)

The Graphical Network Simulator-3 (GNS3) is a network emulator developed by Jeremy Grossmann, Dominik Ziajka and Piotr Pękala released in 2008. Like Packet Tracer, it allows the design and simulation of many modern computer networks, and uses Dynamips to simulate Cisco IOS to achieve this. As such, it provides many of the same features of Packet Tracer, but boasts additional features such as API-support, the ability to run emulated network operating systems, support for containers through Docker, and more, making it an equally appealing choice alongside Packet Tracer.

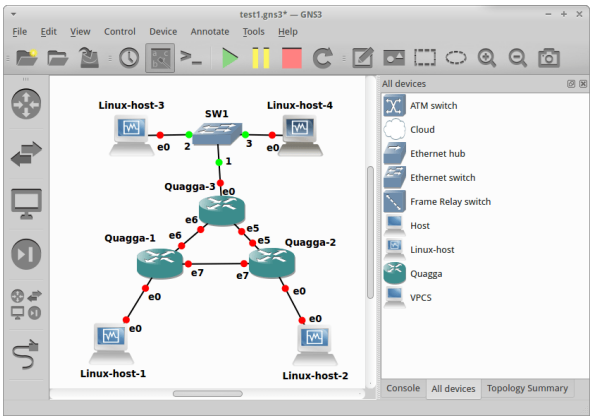


Figure 3.2: Sample Network built in GNS3 (credit: Suketh Evani, Medium.com)

Like Packet Tracer, GNS3 is mainly used in an educational capacity as well as being used professionally, and thus is mainly used for practical applications.

Overall, NDS aim to provide a more practical overview of a network, aiding the user with implementation and thorough testing. They provide details on the different parts used within the network to achieve the desired goal, but is mainly focused on making the network function, rather than providing details on how it functions.

3.1.2 Diagram Design Tools

Diagram Design Tools (DDTs) are pieces of software that allow the creation of visual representations of diagrams, processes, concepts and more. While there are many dedicated DDTs, there are equally as many raster graphics editors that double as DTTs, such as Microsoft Paint, Adobe Photoshop and GIMP.

DTTs provide many features to aid in the construction of various diagram types, such as version control, templates, drag-and-drop functionality, labelling, validation techniques, formatting options, and more. There exists many popular DTTs that are widely used today, such as:

Microsoft Visio

Microsoft Visio is a diagramming and vector graphics application which is part of the Office 365 suite. While it is mainly used to create flowcharts, org charts and mind maps. It can also be used to create network architecture diagrams. As a member of Office 365, Visio can communicate with other Office 365 applications such as Excel to easily integrate data into diagrams. In addition, it provides features such as a Wizard to help with the creation of flowcharts, various diagram templates, diagram validation, and more.

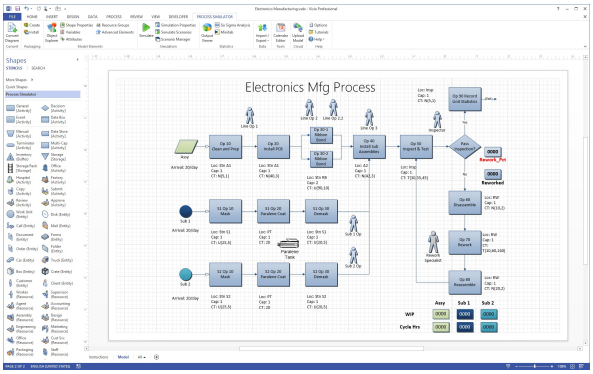


Figure 3.3: A Sample Diagram created in Visio (credit: Lucidchart.com)

While Visio has many more applications besides creating diagrams, it can only provide as much information on a given diagram as what was provided, without any additional information created by the application.

Draw.io

Draw.io, also known as diagrams.net, is a web-based, open-source diagram design application used for creating flowcharts, network diagrams, and other visual representations. One of its biggest features is being a web-based application, meaning that anyone can access and create a diagram without having to download it. It supports many of the same features as Visio, in addition to being extended through plug-ins and other community-made additions, being able to be used without an account, and integration with cloud services such as Google Drive, OneDrive and DropBox.

Similarly to Visio, Draw.io only acts as a tool to construct diagrams and topology and as a result, does not provide any additional information about the diagrams in question. Like Visio, this means all the information for the diagram in question must be sourced by the user separately.

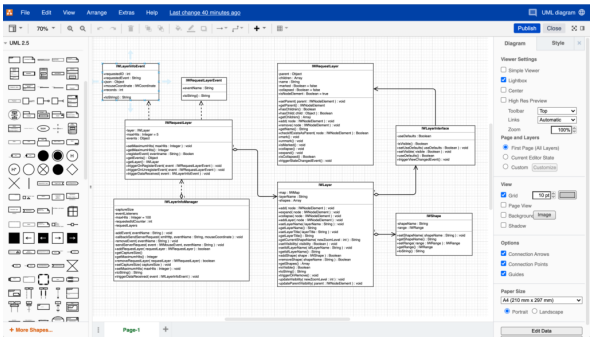


Figure 3.4: A Sample Diagram created in Draw.io (credit: drawio-app.com)

3.2 Analysis of Solutions

The applications listed above provide many use cases and many solutions to a variety of problems. They can be sorted into two main categories based on their functionality:

- **Practical** - where the application is more concerned on the building and testing of a network for possible implementation. The NDS applications (Packet Tracer, GNS3) belong to this group.
- **Theoretical** - where the application focuses more on the outline of the diagram from a top-down perspective, and how it could be improved. Microsoft Visio and Draw.io, as well as most DDTs, would best fit into this group.

Each application has many advantages as well as many disadvantages and limitations that must be considered, especially regarding the design of Clos and Slepian networks and their topology.

3.2.1 Innovative Approaches

The applications above cover a wide array of features that present many advantages for creating Clos and Slepian topology. Some of these advantages include:

- **High Level of Customisation**: As each application can create many different types of networks and diagrams, they all offer many customisation options in order to best suit the needs of the user.
- **Extensibility:** Some applications, like Draw.io and GNS3, can be extended through plugins and other add-ons via the community or by official means, meaning that their feature set can be expanded from what was originally possible.
- **Data-Driven:** Microsoft Visio, for example, allows the integration of data sources into the user's diagrams, meaning that additional information can be used alongside the created diagram.
- **Education Driven:** Many applications are widely used in an educational capacity, to build diagrams ranging from simple implementations to complex networks, and allow experimentation to further advance learning.
- **Ease of Use:** Most of the applications listed above, in addition to many others, focus on being both powerful and easy-to-use, allowing it to be adopted by many users quickly.

- **Creation and Analysis:** The diagrams produced can serve as the foundation for generating additional diagrams, providing more context where necessary, such as with recursive Clos or Slepian networks.
- **Organisation of Information:** Created diagrams can help visualise and organise data generated from other sources, allowing for easier distribution of information.
- **Clarity and Simplification:** Similarly to above, created diagrams help to clarify many parts of collected information and in some cases, provide a simplification on many aspects of the topic.

3.2.2 Limitations

While each application provides a strong list of advantages, there exists an equal number of limitations that could be addressed, to enhance the feature set of the aforementioned design tools. These limitations include:

- **Learning Curve:** While many applications boast an easy-to-use interface, many others, such as GNS3, provide a steep learning curve which may hamper progress.
- **Cost:** Some of the applications we saw, namely Visio, costs money to use, making it less competitive compared to other free-to-use applications.
- **Resource Intensive:** Depending on the size of the created network or diagram, resource use may be a potential concern, and thus must be taken into account throughout development, especially for software such as GNS3.
- **Platform Dependent:** Many applications are exclusive to the Windows operating system, with very limited support for MacOS or Linux, creating another limitation in the application.
- **Simplified Feature Set:** Some tools, such as many DDTs, come with a simplified set of features that may limit the number of possible diagrams that can be created.

The above advantages and disadvantages are not an exhaustive list, but are important when discussing the need for designing Clos or Slepian topology. By noting what other solutions do well and what they can improve upon, we can factor these in when designing our own application to address the limitations, and build upon the advantages that other applications present.

3.3 Justification of Application

As you can see, there exists many different types of applications that could help in the creation of Clos and Slepian topology. However, we can also see that many of the applications mentioned above have a very general use case. To my understanding, no such application exists dedicated to building Clos or Slepian topology. Considering the aforementioned advantages and disadvantages, we can design our application to address some of these limitations, while also enhancing some of the advantages presented.

As outlined in Chapter 1, the aim of the application is to allow the user to construct a Clos or Slepian network topology, and provide information relevant to the network. Additionally, the project also aims to achieve the following:

- **Provide a High Level of Customisation:** During the creation process, I want to provide the user with complete control of the network, from choosing the input to defining the number of stages present in the network. This will include allowing the user to create sub-optimal networks, if desired.
- **Embrace a Data-Driven Approach:** A key aspect of this application is the creation of Clos and Slepian networks, along with the methods used for their calculation and construction. Therefore, I aim to provide users with the option to view the calculations applied to the selected network. This will further extend to resources to help the user understand every part of the created network.
- **Ease-of-Use:** To avoid unnecessary complexity, I will focus on making the application easy to use and accessible for every user. This will ensure that the application can be used and understood by everyone, regardless of their background.
- **Platform Independence:** As the application will be written in the Java programming language, the application can be run on any device that supports Java, which includes Windows, MacOS and Linux machines. For more details on Java, please see Chapter 5, Section 5.3.
- **Light on Resources:** While Java is known to be a resource-heavy language, through clever optimisation techniques and reuse of code, it can significantly cut down on the amount of resources required compared to other programs.

3.4 Related Research Papers, Articles, and Works

The area of computer networking is extremely vast, encompassing many different topics outside of Clos and Slepian networks. There were many research papers, journals and articles that I found to be critical to the understanding of this topic, the creation of the application or, in many cases, both. I believe providing a synopsis of these papers, and their overall impact to the project, is a fundamental step in any field of research. Many problems do not exist in a vacuum, and thus we must showcase where the topics stands at this present moment, and how we plan to iterate on it.

3.4.1 *A study of non-blocking switching networks*

Written by Charles Clos in 1952 and published in 1953, *A study of non-blocking switching networks*[1] describes the origin of the Clos network, and how it could improve telecommunications technology through designing arrays of crosspoints for use in telephone switching systems. This is done by reducing the number of crosspoints such that the establishing of simultaneous connections within the network is not hindered in any capacity. To achieve such a feat, Clos proposed a switching network architecture consisting of multiple stages. These stages would be connected in an hierarchical fashion. As a result, by expanding the number of stages, the networks can achieve non-blocking connectivity through a minimal number of crosspoints, or an "optimal" number. This proved their efficiency in various telecommunications applications.

Clos details that, for any network utilising Clos switches, the number of crosspoints required to achieve non-blocking connectivity is directly related to the input. This also includes the switch dimensions, as well as the number of stages present. The work of Clos is critical to this paper, as it provides every detail required relating to Clos networks. A full breakdown of these findings are detailed in the Chapter 5, where I break down the intricacies of Clos networks in more detail.

Clos' work on switching network architecture marks a pivotal moment in the evolution of telecommunications. The theoretical framework presented in his paper serves as the cornerstone of modern networking principles. As the culmination of extensive research and innovation, Clos' architecture not only offers a solution to the pressing challenges of non-blocking connectivity but also opens new avenues for scalability, efficiency, and reliability in telecommunications infrastructure. The importance of his work, not only to this paper but to modern networking as a whole, should not be understated.

3.4.2 *The Enumeration Method for Selecting Optimum Switching Network Structures*

The Enumeration Method for Selecting Optimum Switching Network Structures (or simply *The Enumeration Method*) is a paper written in 2005 by Polish computer scientists Andrzej Jajszczyk and Robert Wójcik [11]. It delves into a method devised by the authors to find the optimum parameters for an optimum switching network by using a simplistic "enumeration method". This involves the program taking a "brute-force", attempting all possible solutions in a quick and efficient manner in order to find the best solution for the given input. This contrasts to a previous continuous method used originally by Clos to achieve the same results, detailed above. Accompanying the paper is code written in the C programming language that, while initially unavailable, was graciously provided along with permission to modify it by Mr. Wójcik, to which he has my thanks. The code is available through GitHub here.

Primarily focusing on Clos networks, the relevance of this paper is quite significant for a number of reasons. Most importantly, it provides a foundation of code that could be used within the application to calculate the parameters needed to populate the diagram should the user choose the Clos switch. This "enumeration method", as detailed in the paper, focuses around 3 main variables: n, r and m, in order to carry out its functionality. As you will see in Chapter 5: Outline of Approach, Section 5.1, this is not too dissimilar to the variables used in the calculations detailed in that section, with those variables being n, m and $(2n-1)$ in place of n, r and m, respectively. While I will delve into the inner workings of the code, as well as some of its flaws and how it can be improved, it is recommended to read the above section to fully grasp the concepts explored below.

The code is split into 3 main files, to accommodate a 3, 5 and 7-stage Clos network. The 3-stage code can be seen in Figure 3.5. It begins with asking the user for the total number of inlets, N (which would be the total inputs/outputs). From here, starting with $n = 1$, the code enumerates over each value of n, such that $1 \leq n \leq N$. From here, the code calculates a value for r, depending on the criteria met: should n % $N == 0$, then $r = N/n$. Otherwise, $r = floor(N/n)$. This is to accommodate any decimal values that may exist, instead rounding to the "floor" of the whole number. (I.E., should N/n be equal to 22.4, for example, r would then be equal to 22, and not 23.) From here, it would then calculate m, which is simply $m = 2n - 1$, before calculating the number of crosspoints present in the network, as detailed in Chapter 5: Outline of Approach, Section 5.1. Following this, the code will then check the result against the values currently noted as providing the lowest number of crosspoints for the given input. Should the newly calculated crosspoint count be lower than what is stored, the numbers are overwritten. Following this, the while loop begins again until $n = N$, where the values calculated are returned.

```
//      Program evaluating optimal 3-stage Clos network
//      structure for a given total number of inlets.

#include <stdio.h>
#include <math.h>
#include <conio.h>

int main(void)
{

long C3, C3min;          // "long double" for very large numbers
int m, n, r, N, ropt, nopt;

n=0; C3min = 0;                // Initial values

printf ("\n\nGive total number of inlets: ");
scanf ("%d", &N);              // Getting total number of inlets

while (n++ < N)                // Count every possibility n = 1,2...N
{
if (N % n == 0) r = N/n; else r = floor(N/n) + 1;
      // Count r as lowest possible ensuring AT LEAST the total N inlets.
m = 2*n-1;                     // Count m = 2n-1

C3 = 2*r*m*n + m*r*r;
      // Count total number of crosspoints C3 = 2mnr+mr^2
      // Casting (long double) ensures compilator uses this type
      //printf ("\nOutside If Testing (n %d, r %d) consists of %d crosspoints. ", n, r, C3);
if ((C3 < C3min) || (C3min==0))        // If counted C3 is minimal whatsoever
{                                      // save Clos network parameters
       C3min = C3;
       ropt = r;
       nopt = n;
       printf ("\nTesting (n %d, r %d) consists of %ld crosspoints. ", n, r, C3);

};

};

printf ("\n\n3 - stage Clos network\n\nn - number of inlets per switch in section 1,\nr - number of switches in section 1");
printf ("\n\nThe optimal switching network (n=%d, r=%d) consists of %ld crosspoints. ", nopt, ropt, C3min);

printf ("\n\nPress any key to continue.");

getch();                               // Waiting for a key - to see results
return 0;
};
```

Figure 3.5: Code detailing the 3-Stage Clos network Enumeration Method. (credit: Robert Wójcik)

The code diverges slightly for the 5-stage and 7-stage calculations. The middle stage of the network (denoted by mrr, as seen in Figure 3.5), is instead replaced by a recursive call of a $k-2$-stage network, where k equals the number of stages in the complete network, as detailed in Chapter 5: Outline of Approach, Section 5.1. This recursive call is an almost identical piece of code to the code for the number of stages it represents, with the difference being the code returns the number of crosspoints found by the lowest count as an integer value, which is then used in the 5-stage network. An overview of the code for the 5-stage Clos network can be seen in Figure 3.6.

The code for the 5-stage network, and indeed the 7-stage network succeeding it, is almost identical to that of the 3-stage network observed previously. The main difference is that the middle stage is replaced with a recursive call for a 3-stage Clos network to represent the middle stage, as mentioned before.

Results obtained from this approach, as detailed in *The Enumeration Method*, are extremely impressive for its time. For $N = 1000$, we can see execution times of at most $0.02s$, and for $N = 1000000$, the execution times took as little as $0.52s$ for a 3-stage Clos network, and $32.60s$ for a 7-stage network. It is important to note that these results were obtained in 2004, and in the 20 years since the paper's publication computer processors have evolved quite substantially. Due to this, I conducted the same tests locally, to gain a better understanding of the code's optimisation today. These tests were conducted using an Intel Core i7-10750H processor clocked at 2.60GHz, using a PC611 NVMe SK Hynix 1TB solid state drive (SSD), along with 16GB of DDR4 SODIMM RAM, using the Windows 11 OS. A table comparing the results obtained to the results seen in the paper can be found in Table 3.1.

Comparing the results obtained today to the original results showcases the grand leap that tech-

```
main ()
{

long C5, C5min;
int m, n, r, N, ropt, nopt;

n=0; C5min = 0;

printf ("\n\nGive total number of inlets: ");
scanf ("%d", &N);

while (n++ < N)
{
if (N % n == 0) r = N/n; else r = floor(N/n) + 1;

m = 2*n-1;

C5 = 2*r*m*n + m*Clos3(r);
      // Instead of middle section 'r*r' there is a 3-stage structure
      // Function Clos3() returns minimal number of crosspoints for given 'r'
      //printf ("\nTesting outside If C5 (n %d, r %d) consists of %ld crosspoints. ", n, r, C5);

if ((C5 < C5min) || (C5min==0))
{
        C5min = C5;
        ropt = r;
        nopt = n;
        printf ("\nTesting in If C5 (n %d, r %d) consists of %ld crosspoints. ", n, r, C5min);

};
};

printf ("\n\n5 - stage Clos network\n\nn - number of inlets per switch in section 1,\nr - number of switches in section 1");
printf ("\n\nThe optimal switching network (n=%d, r=%d) consists of %ld crosspoints. ", nopt, ropt, C5min);

printf ("\n\nPress any key to continue.");

getch();                              // Waiting for a key - to see the results
return 0;
};
```

Figure 3.6: Code detailing the 5-Stage Clos network Enumeration Method. (credit: Robert Wójcik)

	TIME (seconds)					
STAGES	3		5		7	
YEAR	2004	2024	2004	2024	2004	2024
INPUT						
1000	<0.01	0.000175	<0.01	0.000224	0.02	0.000838
10000	0.01	0.000392	0.03	0.001219	0.18	0.005770
100000	0.05	0.000869	0.42	0.011896	2.50	0.077262
1000000	0.54	0.004509	4.99	0.542131	32.60	10.378504

Table 3.1: Table comparing results from *The Enumeration Method* vs results obtained using the same code in 2024

nology as a whole has taken since the paper's publication, especially in the case of a 7-stage Clos network. This saw the biggest improvement, with the time for an input of 1000 improving by over 95%. This gives us a better understanding of the usability of the code in relation to the speed of execution. It is safe to say that the code very much holds up today in 2024.

The Enumeration Method is an extremely impressive publication for 2005, as its optimisation techniques were quite beneficial to the area of switching networks at the time. Despite this, I believe that this method can be improved further. As detailed in Chapter 5: Outline of Approach, Section 5.3, I have made a number of improvements and modifications to the code that not only improve its speed further, but reduce the code's footprint from multiple methods across 3 files to only 2 methods total. Additionally I have modified the code to use it as a basis for a Slepian switch, achieving similar results. For a full breakdown of these improvements and modifications, please see Chapter 5: Outline of Approach, Section 5.3.

Overall, *The Enumeration Method for Selecting Optimum Switching Network Structures* serves as a critical work to this project, providing a code basis for which the application builds upon. It breaks down how the authors improved upon the methods outlined in Clos' original paper and

showcases the results of that research. Thanks to the generosity of Mr. Wójcik, I have been able to obtain, study and improve on the original paper's goal, as well as expand it to other network switches, most notably the Slepian switch. This expansion has proved extremely fruitful to the application, as described in Chapter 5: Outline of Approach, and paved the way for the creation of a paper that describes these advancements, titled *Enumerating the Enumeration Method: Further Enhancements for Selecting Optimum Switching Network Structures*.

Chapter 4: Project Work-plan

Given the scale of the project, it is important that a solid, yet flexible plan is put in place. It ensures that milestones are met on time. In the event that a deadline is missed, it can allow plans to be rerouted to ensure the most pressing tasks are completed to a high standard, such that the integrity of the project is not compromised. I have created two specific Gantt Charts to represent this: one for the period of 11th September 2023 - 31st December 2023 (denoted as **P1**), and another for the period of 22nd January 2024 - 26th April 2024 (denoted as **P2**), representing the time frames in which this project will be carried out.

4.1 Period 1: 11/09/23 - 31/12/23

The first Gantt Chart, for the period P1, is as follows:

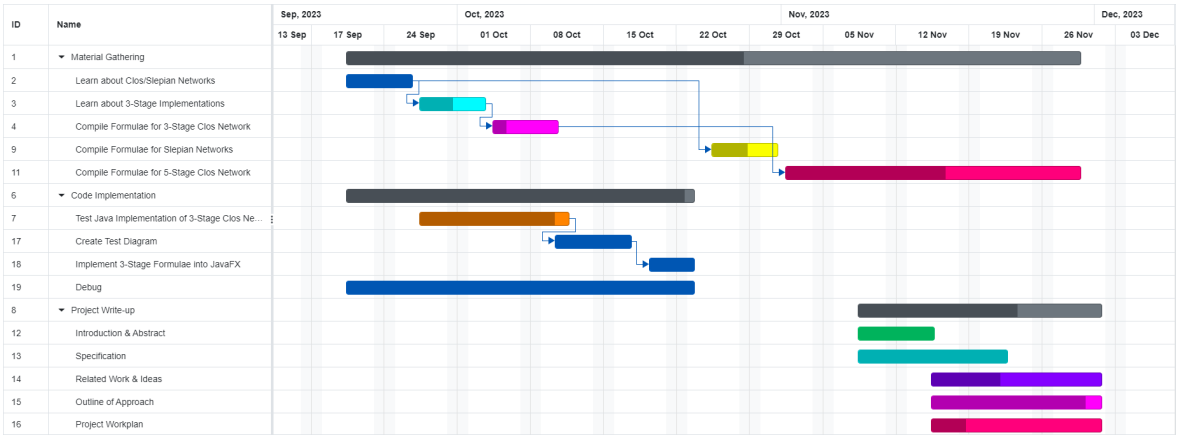


Figure 4.1: Gantt Chart for P1

The tasks contained in the above chart are sorted into 3 main groups: **Material Gathering**, **Code Implementation**, and **Project Write-up**. It allows complimentary tasks and objectives to be grouped together for easy viewing, and showcase dependencies between tasks where relevant. The breakdown of tasks, by group, is as follows:

Material Gathering

- **Clos/Slepian Networks Familiarization:** As Clos and Slepian networks were entirely new topics to me, my initial priority was to familiarize myself with these networks and comprehend their construction, applications, and implementation.
- **Understanding 3-Stage Implementations:** Following the above, my next task was to delve into the specifics of 3-stage networks, serving as a foundational understanding for future networks.

- **Compilation of 3-Stage Clos Networks Formulae:** To prepare for experimentation in Java implementation, I embarked on researching the formulae utilized in 3-stage Clos networks and understanding their translation into a Clos topology.
- **Compilation of Slepian Networks Formulae:** Slepian networks, being based on the 3-stage Clos network, share similar formulae. Compiling a formula for a 3-stage Slepian network would establish a solid groundwork for larger Slepian networks, which will be explored further in P2.
- **Compilation of 5-Stage Clos Networks Formulae:** Given that 5-stage networks are extensions of 3-stage networks, I initiated the compilation of formulae for 5-stage networks. This information gathering lays the groundwork for future recursive networks, particularly the 7-stage Clos network.

Code Implementation

- **Testing Java Implementation of 3-Stage Clos Networks:** Upon completing the compilation of formulae for 3-stage networks, I proceeded to create a test class in Java. This class implements the aforementioned formulae, allowing for the testing of calculations and comparison with provided examples, thereby providing a foundation for future network implementations.
- **Creation of Test Diagram:** Subsequently, I created a test diagram representing a standard 3-stage topology. This step aimed to familiarize myself with JavaFX and assess the extent of work required for the application's programming.
- **Integration of 3-Stage Formulae into JavaFX:** Building upon previous steps, I consolidated my work into a single project. Before this, I segregated my work into individual projects for testing and research purposes. The third project, serving as a test application, incorporates the 3-stage Clos class and the previously developed test topology, linking the two. This integration facilitates the printing of results onto a topology created in JavaFX, presenting an intriguing challenge.
- **Error Resolution:** Throughout the development of any application, encountering bugs is inevitable. Therefore, I allocated additional time to address any arising issues or rewrite malfunctioning code, ensuring the application operates as intended.

Project Write-up

- **Introduction and Abstract:** The initial task involves drafting an introduction to the project, delineating the concepts of Clos and Slepian networks, the rationale behind the project, its intended progression, and an abstract providing a succinct overview of the paper's overarching goal.
- **Specification:** Another critical task entails discussing the project's specifications, outlining its objectives, goals, and any other pertinent information.
- **Review of Related Work and Concepts**: Delving into existing literature and similar applications, I intend to identify their strengths and weaknesses. Furthermore, I will briefly outline how I plan to address these limitations and leverage the advantages of each.
- **Approach Outline:** During this phase, I will formulate a chronological outline detailing how the project will unfold.
- **Project Work-plan:** This section will feature a detailed week-by-week outline of my plans, accompanied by Gantt charts to complement the information presented.

4.2 Period 2: 22/01/24 - 26/04/24

The second and final Gantt Chart, for the period P2, is as follows:

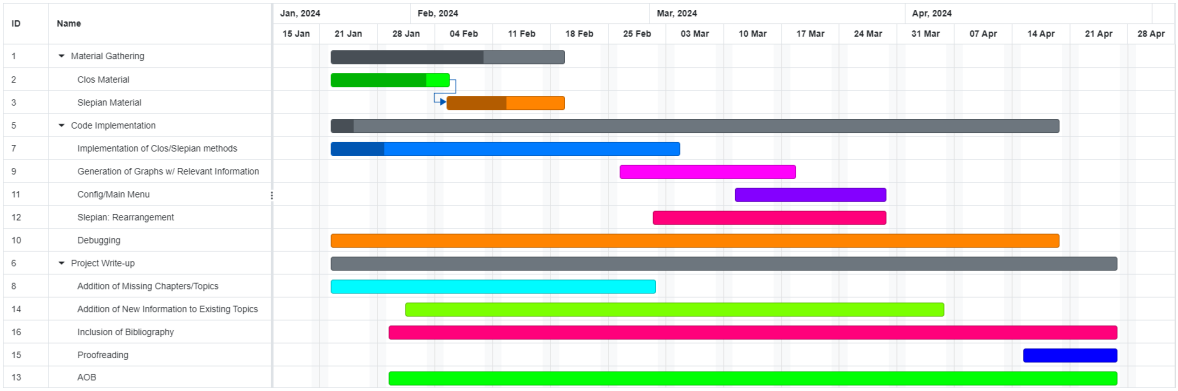


Figure 4.2: Gantt Chart for P2

The main groups from Period 1, being **Material Gathering, Code Implementation,** and **Project Write-up**, have been carried over to Period 2, as they worked well during Period 1 of this project. However, new tasks have been added to this group to reflect the work completed from Period 1, as well as any additional tasks that have been added as a result. The breakdown of these tasks are as follows:

Material Gathering

- **Clos Compilation:** With most of the required information acquired in P1, the main objectives will be to simply organize and cross-reference the material to ensure its validity and accuracy. While minor, this step is crucial, as verifying the accuracy of information is paramount. Additionally, further information regarding the theory of recursive Clos networks was sorely needed, and a significant amount of time will be allocated to delve into this topic.
- **Slepian Compilation:** A substantial amount of time in P1 was dedicated to studying Clos networks, primarily due to Slepian networks existing as a variant of them. At its conclusion, I had gathered the formulae and the most pertinent information for 3-stage Slepian networks. However, additional research was necessary - specifically concerning recursive Slepian networks - and how this could be integrated into the application.

Code Implementation

- **Clos/Slepian Method Implementation:** The implementation of multistage Clos networks underwent testing during P1, partly due to the code provided by Mr. Wójcik and his implementation in C. However, this existed as five separate C methods, and thus, I aim to explore if this could be refactored and consolidated into at most two methods. For the Slepian implementation, I aim to develop methods based on the information obtained above and ensure it performs comparably to the Clos methods, in terms of execution time.
- **Graph Generation with Relevant Data:** During P1, I successfully implemented an example of a 3-stage Clos network on a sample graph using JavaFX. However, it was largely "hard-coded" and not conducive to expansion without significant refactoring. As such, I will begin outlining a plan for the final build's structure. This will entail how information from the aforementioned methods will be conveyed to the final implementation of the main graph, using the prototype previously developed as a guide rather than a basis. This would

necessitate reconstructing the graph within JavaFX, enabling adaptability with both Clos and Slepian methods, as well as accommodating information from various sources.

- **Menu Design:** Designing a menu that is user-friendly while also presenting all available options to users is challenging. Designers must consider various factors, such as navigation flow, feedback and confirmation, and readability, among others[12]. I anticipate testing various menu prototypes throughout P2, for both the Configuration Menu (where users input information for the intended graph) and the menu displayed upon executing the program, to ensure clarity, consistency, and ease of use. The techniques employed here will then serve as a foundation for any future menus within the program.
- **Slepian Network Rearrangement:** A characteristic of Slepian networks is their ability to be rearranged provided they satisfy the condition that $m \geq n$. [4]. While not strictly necessary, I would like to attempt to demonstrate this rearrangement of the diagram, providing an interactive element to the application.
- **Error Resolution:** This time is allocated to resolving any bugs that may arise upon completing the full application, ensuring it runs as effectively and efficiently as possible.

Project Write-up

- **Missing Chapter/Topic Addition:** At the end of P1, many chapters were included but did not contain any meaningful content, or were omitted entirely. During this period, I aim to finalize the layout of this paper and incorporate these missing sections.
- **New Information Incorporation:** The primary focus of P1 was on the Clos switch, as previously mentioned. Consequently, much information - particularly regarding Slepian switches - was omitted. Upon adding the missing topics above, I intend to incorporate new findings regarding Slepian networks, as well as updated information on Clos networks.
- **Bibliography Inclusion:** A significant issue encountered was the rendering of the bibliography, where sources were cited but no index of these sources was included. I aim to rectify this before completing this paper.
- **Proofreading:** Similar to the "Error Resolution" section above for the program, I intend to spend a considerable amount of time "proofreading" the paper to ensure that the presented information is accurate and factual, and that the paper is free from mistakes and flows as intended.
- **Any Other Business (AoB):** Should any issues arise that do not fall into the above categories, I have reserved this time to ensure these issues are addressed.

Chapter 5: System Design

With the many graph and diagram drawing applications that currently exist, many of them fail to provide innate support for Clos or Slepian network design, specifically for detailing information relating to the given network. Many support the free creation of a diagram drawing which, while useful for a variety of cases, does not provide any information except the information that is manually entered. This need for detailing the specifics of a Clos or Slepian network in a single place became the starting point that was needed: to create an application that draws the diagram, and provides the relevant information about that specific network. As a result, this gives us an indication of what's required: what is a Clos or Slepian network? What is required to create one? What are the related formulae? In relation to the diagram, what are the different parts of the created network? How does a Slepian network differ from a Clos network? Are there any additional formulae or considerations that must be taken into account? Before work on the application can begin, these questions, and more, must be answered.

5.1 Assembling the Required Formulae

The first step in this approach was understanding the concepts behind Clos and Slepian networks, alongside assembling the formulae associated with it, in order to begin the design of what was required of the application. The best starting point for this was Clos' paper, *A study of non-blocking switching networks*. It provides information as to the reason for the development of Clos networks, as mentioned previously, but most importantly it provides details on the calculations required. Additionally, *The Enumeration Method for Selecting Optimum Switching Network Structures* served as an aid alongside it, providing a basis for the implementation of the code for the Clos network, which I later expanded to the Slepian network. These papers, as well as their significance to this paper, can be viewed in more detail in Chapter 3: Related Work and Ideas, Section 3.4.

Clos Networks

The formulae provided within Clos' paper gives us a good indication of what needs to be consider for the application. One such note that needs to be taken into account is that there exists slightly different calculations depending on the number of stages that exist within the network, but it should be noted that the differences are minor. For example, the calculations for finding the minimum number of crosspoints for a 3-stage Clos network is different to that of a Clos network with a stage size greater than 3. This is due to the innate fact that the middle stage of any k-stage network contains a smaller $(k-2)$-stage network itself. However, these calculations are not complex by any means. Instead, we can adapt the calculations used for finding a 3-stage network, for any k-stage network, as was done in *The Enumeration Method*, and as detailed in Chapter 3. Such types of networks can simply be referred to as "Recursive Networks", due to the recursive characteristics of the middle stage.

Figure 5.1 showcases a square array with N inputs, N outputs, and N^2 crosspoints. This showcases that, for any N inputs and N outputs, the minimum number of crosspoints required such that there does not exist any blocking between the inputs and outputs will always be N^2 in this particular

$$C(1) = N^2$$

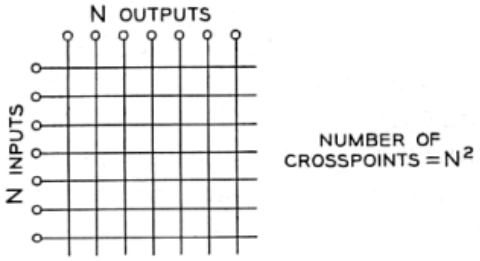


Figure 5.1: A square array representation showcasing N inputs and outputs[1]

case[1]. For example, should there be $N = 10$ inputs and $N = 10$ outputs, there will exist $N^2 = 10^2 = 100$ crosspoints within the topology. While this is true for the above, where there is a single stage, it is not necessarily true for later networks where more stages exist, however it should be noted that one constant across all networks irrespective of stage count is the fact that there will always be N inputs and outputs.

3-Stage Clos Network

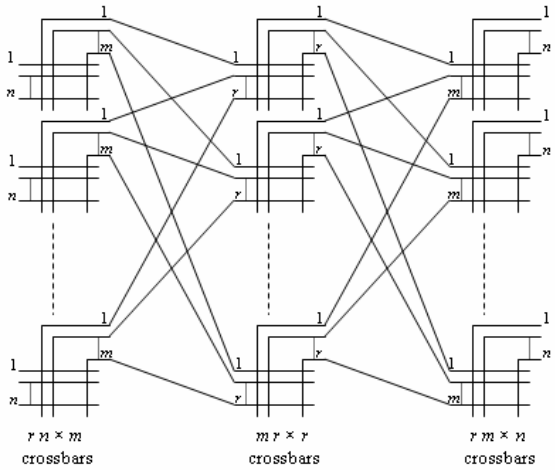


Figure 5.2: 3-stage Clos Network[1]

For a 3-stage Clos network, as depicted in Figure 5.2 above, a different calculation for finding the minimum number of crosspoints is required for this particular network. The number of crosspoints that exist within a k-stage network is denoted as $C(k)$. For a 3-stage Clos network, the number of crosspoints is usually $C(3) <= N^2$,[13] however in most cases there will not be N^2 crosspoints present. To calculate $C(3)$, the optimum number of crosspoints required for a given N, we use the following formula:

$$C(3) = 2mn(2n-1) + (2n-1)mm[1]$$

where $mn = N$, and both serve as the optimum values that give N as their product. These two values are required for all calculations for any stage, and can be calculated from N. m and n can be found as follows:

$$m = \sqrt{\tfrac{N}{2}}[14]$$

$$n = \sqrt{2N}$$ [14]

These formulae are constant across all Clos networks, regardless of stage count, unlike with the formulae with finding the optimum crosspoints. When finding values for n and m, we must be wary of decimal values. Take the following example, with $N = 2000$:

$$m = \sqrt{\frac{2000}{2}} = 31.623$$

$$n = \sqrt{2(2000)} = 63.246$$

We note that the values above have decimal values. In such an event, **we take one of the values, and pick another value such that the product is at least 2000.** For example, should we wish to take m, we could round up to $m = 32$, and take a value for n such that $N <= mn$. In one such case, we could take $n = 63$, and have it such that $N = mn = 2016$, meaning that 2016 is the new value for N, as it is the closest approximation that will be allowed using the calculated m and n. Another example is taking $m = 31$. In this case, we must take $n = 65$, meaning $N = mn = 2015$, taking 2015 as the new value for N for the reasons stated previously. We should attempt to find a value that is closest to the original N value, but in many cases this may not be possible[14].

This should be noted that it is indeed possible to simply choose a value for n or m, and not use the provided formulae above. However, we must ensure it meets certain criteria. For this example, let us define x as the chosen value for either n or m, and define y as the other value. These checks are as follows:

$$1 \leq x \leq N$$

$$x \neq y$$

$$x = N/y$$

In essence, we must ensure that for any value we choose passes all the above criteria. It is important to note that failing $x \neq y$ means that the network results in a Slepian network.[4] We will delve into this topic in more detail later.

The final required formulae are those that calculate the number of switches in a given stage, as well as the crosspoints per switch in each stage. In Figure 5.2 above, we showcased a 3-stage Clos topology for N inputs and outputs. As part of that topology, it showcased the number of switches that are present in each stage, as well as the number of crosspoints present in each switch. For these calculations, we must use different calculations depending on if we are working with Stage 1 and Stage 3, or Stage 2. This is because the architecture of Stage 2 is different as it must route the inputs correctly to the outputs, while Stage 1 and Stage 3 share an identical, albeit mirrored, architecture.[1]

To find the number of crosspoints in each switch in a network with N inputs and outputs, we must first define values for m and n through either method mentioned previously. When these values are known, we use the following formulae, once again depending on the stage:

Stage 1: $n(2n - 1)$

Stage 2: m^2

Stage 3: $(2n - 1)n$

Note that the formulae for Stage 1 and Stage 3 are indeed identical, only mirrored. Concerning the number of switches in each stage, we must once again use the values m and n in our calculations. The formulae for finding the number of switches in each stage are as follows:

Stage 1: m

Stage 2: $2n - 1$

Stage 3: m

Once again, the formulae for Stage 1 and Stage 3 are identical. For a 3-stage Clos network, these formulae are all that is required to construct the topology we desire. As mentioned previously, should we wish to construct a 5-stage, 7-stage or above network, we need to modify the middle stage in order to transform our network into a Recursive Clos network to further optimise our network and handle larger inputs.

Recursive Clos Network

Recursive Clos networks are simply modified 3-stage Clos networks in which the middle stage is replaced with a $(k-2)$-stage Clos network[15]. Indeed, this allows us to expand the scope of what was capable with the original 3-stage Clos network design, and optimising the system further for larger input sizes. It is also possible for there to be another 3-stage Clos network as the middle stage of network already contained within another network, which allows for the expansion to larger networks[16].

The topology that represents a recursive Clos network is fundamentally identical to that of a 3-stage Clos network. When viewing the topology, it should be noted that the middle stage, prior to closer inspection, accounts for every stage that is between stages 1 and k, and thus appears to be significantly larger than either stage. In relation to calculating the recursive network's switch dimensions, number of switches per stage, and values for m and n, they are indeed identical to that of a normal 3-stage Clos network, and thus we can use the formulae we observed previously. The core change comes in the form of how the crosspoints are calculated.

As the middle stage is itself a Clos network, it is not possible to simply use the 3-stage crosspoint formula to calculate the total crosspoints. Instead, we must calculate the crosspoints for each network within the overarching network. For example, should we have $k = 7$, we must calculate the crosspoints for the 7-stage, the inner 5-stage, as well as the 3-stage network contained in that. This is done by replacing the middle stage part of the 3-stage formula with the total crosspoints for the $(k-2)$-stage formula. Observe the formula we noted previously:

$$C(3) = 2mn(2n-1) + (2n-1)mm[1]$$

The section of the formula highlighted in red denotes the section responsible for calculating the number of crosspoints within the middle stage. In order to correctly calculate the crosspoint count for the recursive network, this must be modified as mentioned previously. This is done simply by replacing the middle stage section to account for the inner networks, as follows:

$$C(k) = 2mn(2n-1) + 2n-1(min(k-2, m))[11]$$

In the above, we replaced a portion of the middle section with $min(k-2, m)$ to account for the crosspoints of the inner network. In the above, k represents the total number of stages, with $k-2$

representing the inner network. We use m of the overall network as the "input" for the inner network, meaning that for the $(k-2)$-stage network, $m = N^1$, with N^1 denoting the N of the inner network. From here, we apply the previously seen material to this inner network.

It should be noted that the calculation for $min()$ changes depending on the value of k^1, with $k^1 = (k-2)$, and is as follows:

If $k^1 = 3$:

$$C(k^1) = 2mn(2n-1) + (2n-1)mm$$

If $k^1 > 3$:

$$C(k^1) = 2mn(2n-1) + 2n-1(min(k^1-2, m))$$

This repeats until $k^n = 3$, in which you use the standard formula for a 3-stage Clos network, as above.

The number of stages to use within a recursive network is based entirely on the input, N. Within the application, a brute-force approach was taken whereby the optimal crosspoints were compared to succeeding and preceding stages, starting with $k = 5$. During comparisons, a point will be reached whereby a greater number of stages will produce a larger crosspoint count for an input N.[1]. Denoting this stage count as X, the optimum number of crosspoints will be $X-2$-stages for an input N.

Let us illustrate the above with an example. Observe a 7-stage Clos network with $N = 2506, n = 14$ and $m = 179$. As it is a 7-stage network, the middle stage is a 5-stage network, which itself contains a 3-stage network. Passing m as the "input" for the 5-stage network, and m^1 (which is m for the 5-stage network) as the "input" for the 3-stage network results in the following table of variables:

	STAGE		
VARIABLES	7	5	3
N	2506	179	30
m	179	30	10
n	14	6	3
2n-1	27	11	5

Table 5.1: Breakdown of variables within the sample Recursive Network

This showcases the full breakdown of variables. As noted previously, the value for m indeed becomes the value of N within the inner network. With these variables in hand, we can now formulate the equations we will need to calculate the crosspoints, as follows:

$$C(3) = 2(10)(3)(5) + (5)(10)(10) = 800$$

$$C(5) = 2(30)(6)(11) + (11)(C(3)) = 12760$$

$$C(7) = 2(179)(14)(27) + (27)(C(5)) = 479844$$

Thus, the total number of crosspoints for a 7-stage Clos network with $N = 2506$ is 479844. Additionally, we also have all the information of the inner networks of the 7-stage network, allowing us to expand the network and showcase as much information as required. For a 9-stage network or greater, you follow the same procedure, ensuring you also calculate the crosspoint counts for the networks that succeeded the 7-stage network.

Slepian Networks

3-Stage Slepian Network

While we primarily focused on the calculations revolving Clos networks to this point, this was mandatory to understanding the calculations behind a Slepian network. Indeed, the very nature of the Slepian network is that it is simply a variation of the Clos network[4], and thus shares quite a number of similarities, as we have discussed extensively so far. This includes how the number of crosspoints are calculated, finding the most optimum values for the variables of a Slepian network, n and r, and the recursive nature of any network greater than 3-stages.

An important starting point would be to discuss the calculations of the number of crosspoints found in a 3-stage Slepian, and follow the path we explored in a 3-stage Clos network. For any k-stage Slepian network, there exists 2 formulae that can be used to calculate the total number of crosspoints, with one being similar to the 3-stage Clos network. This formula is as follows:

$$C(3) = 3(2((2r)n) + (2n)r)[17]$$

where $rn = N$, and both serve as the optimum values that give N as their product within a Clos network, in the very same manner that m and n serve as the optimum values that give N. Finding these two values are identical to that of a Clos network, being:

$$n = \sqrt{\frac{N}{2}}[18]$$

$$r = \sqrt{2N}$$

Once again, these can be used for any k-stage network. When finding values for n and r, we must take precautions should the values result in decimals. Taking the example of N = 1024:

$$n = \sqrt{\frac{1024}{2}} = 22.63$$

$$r = \sqrt{2(1024)} = 45.25$$

As we noted before, **in this event we take one of the values, and pick another value such that the product is at least 1024.** For example, we can take $n = 23$, and take $r = 45$. This would give a product of 1035 as $(45)(23) = 1035$, which would be the closest approximation to 1024 that we can create.

Similarly to the Clos network, it is indeed possible to manually select values for n or r without using the above formulae. In fact, manually selecting a value for r is the basis for a Fixed Switch-Size Slepian Network. As before, there are certain criteria that must be met in order to do so. Let us denote x as the chosen value, and y as the other value. These checks are:

$$1 \leq x \leq N$$

$$x = N/y$$

As you can see, they are virtually identical to that of which we observed previously, with the only difference being the removal of the $x \neq y$ check, as a correctly optimised Slepian network will always have $n = r$ at its innermost network.[17]

When we observed the Clos network, we noted that depending on the stage, there would be different ways of calculating the dimensions of the switch. In the case of the Slepian network, this process is streamlined significantly. Each switch of the Slepian network, regardless of the stage, is made up of the same dimensions. (cite optimal structures of benes networks). As a result, the switch dimensions of all switches within the network is as follows:

Switch Dimension of a Slepian Switch: r^2[19]

Not only does this make calculating the switch dimensions uniform, we will see later on that **this helps to simplify calculating the total number of crosspoints significantly,** allowing us to optimise the calculations of a Slepian network even further. However, it should be noted that the above is true only if you were to observe the full, unfolded diagram. In the case of the recursive network, this will change, and the middle switch will have a different switch dimension. We should also draw attention to the fact that this different switch size simply accounts for every r^2 switch that exists within the recursive network. We will delve into this in more detail when observing recursive Slepian networks below.

We also noted that with a Clos network, as well as the switch size, the number of switches present in a stage depends on which stage is being observed. Similarly to each stage using identical Slepian switches, each stage has an equal amount of switches in the stage, denoted as the following:

Switches per Stage: n[19]

As mentioned previously, this will vary slightly in the case of recursive networks. However, we will delve into these differences, and more, now.

Recursive Slepian Network

Similarly to a recursive Clos network, a recursive Slepian network is a large Slepian network that is represented as a modified 3-stage Slepian network. That is, the middle stage is itself a $(k-2)$-stage network, where k is the total number of stages present in the network. As a result, there are a number of important changes to note when dealing with such networks.

Almost identically to a Clos network, the middle stage of a k-stage Slepian network is simply a $(k-2)$-stage Slepian network, taking the value of n from the previous network as its "input"[20]. For example, if we observe a 15-stage Slepian network with $N = 6561, r = 3$ and $n = 2187$, the network that would encompass stages 2 - 14 would be a 13-stage network with $N = 2187, r = 3$ and $n = 729$. Following this, the network showcasing stages 3 - 13 would be an 11-stage network with $N = 729, r = 3$ and $n = 243$. This pattern continues until the inner most 3-stage network has $r = n$, which would be 3 in this case. Note that as we explore these inner stages, while the switch dimension remains constant, the number of switches per stage decreases, until stages $(k-1)/2, k/2$, and $(k+1)/2$, where $n = r$. After stage $(k+1)/2$, the number of switches per stage begins to increase again.

As we can see above, no matter the size of the network, each switch will always have a dimension of r^2. However, simply displaying this as the dimensions for the middle stage of a network with $3 \leq k$ -stages would be falsifying the topology. Understanding how to represent the inner-networks of the middle stage is a key step. By observing the results obtained from testing as seen in Chapter 6, we note that only the smallest inputs for a Slepian network, when optimised, result in a 3-stage network, meaning that unless the user creates an unoptimized network, the resulting network will almost always be a 5-stage network or greater.

As we discussed previously when detailing a 3-stage Slepian network, we noted 2 key variables, being n and r, as well as their significance to representing the topology. When observing a recursive Slepian network, the use cases of these variables remains largely the same, except in the case of the middle stage of the topology, where the following is true:

Middle Stage Switch Dimension: n^2[4]

Switches per Middle Stage: r[4]

We can see that the roles of the variables have effectively reversed, with the middle stage dimensions being n^2 as opposed to r^2. While this may seem like a large change, in reality nothing has changed at all, as we noted previously that no matter the number of stages in the network, the switch dimensions will always be r^2. The reason for the dimensions of the middle switch appearing as n^2 is simply because **the middle switches represent every single r^2 switch that is within the recursive network as a collection of much larger switches**[21]. Indeed, by folding out the network we can see that this is true, as we observed in the example previously.

To calculate the total number of crosspoints for the recursive network, we can take a similar approach to that of the Clos network, where we continuously calculate the crosspoints of all the inner networks before adding these values together. While this approach works, we can instead opt for a much simpler formula, utilising the stated fact that each stage contains the same number of switches, as well as each switch having the same dimension. The formula is as follows:

$$C(k) = (r^2)nk\text{[17]}$$

where k is the number of stages in the network, r is the dimension of the switch, and n is the number of switches in a stage, as seen previously. This calculation is can be used for any k-stage network, including 3-stage Slepian networks, meaning it can be used in place of the formula we observed prior. This formula is the one that was ultimately used within the final build of the application, with more detail as to why that is being found in the Code Implementation section.

To find the optimal number of stages for a given input N, we must use the calculated variable r. Ideally, the number of inputs should be a power of r, such that the following criteria is met:

$$r^x = N$$

where x is the power that r is raised to. The number of stages is then $2x - 1$. For example, should we have $r = 2$ and $N = 1024$, the ideal number of stages would be 19. This is because $r^{10} = 1024$, meaning $x = 10$, which in turns means $2x - 1 = 20 - 1 = 19$.

Fixed Switch-Size Slepian Networks

A fixed switch-size Slepian network, more commonly referred to as a Beneš network[22], is a Slepian network of any size whereby we state the value of r manually and construct the network as normal. The most common values for r used is 2 or 3, however larger values for r, while rare, are also used. Should we choose to set the value for r, we can calculate the value for n as follows:

$$n = N/r$$

5.2 Understanding the Topology

The topology is the diagram that describes the Clos or Slepian network, and all the interconnected parts associated with it[1][22]. It provides a clear overview of the network, mentioning most of the information detailed above, specifically the N inputs and outputs, the number of switches in each stage, as well as the dimensions of each switch. The topology is relatively easy to construct manually, with the key details being the switches, the connections between each switch, as well as the inputs/outputs on the first and final stages. A sample topology, constructed using the application this report is based on, is seen below while also pointing out important details mentioned previously

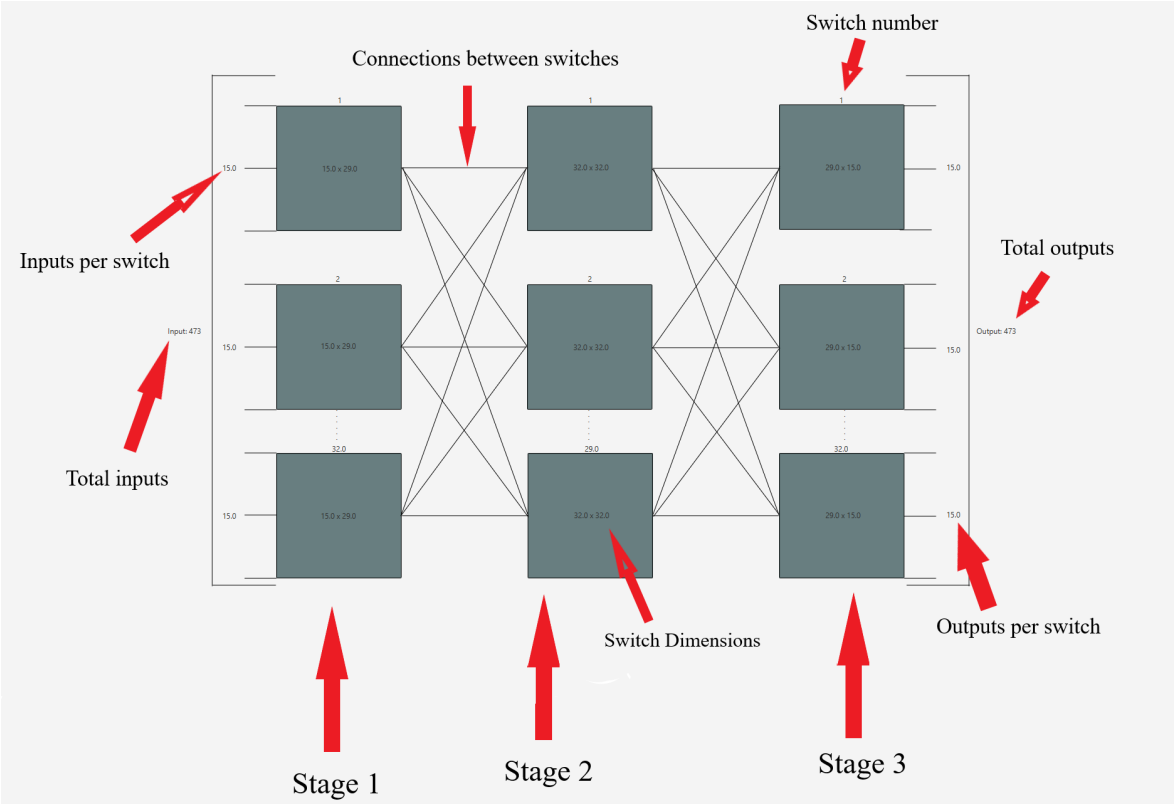


Figure 5.3: Overview of a typical Clos/Slepian Topology

The topology present is applicable to both a Clos and Slepian network, as they share a common topology. As mentioned previously, calculating the specific details change based on the type of network produced. The above topology can indeed represent a recursive network too, with the middle stage representing all the switches and stages within that network.

Building the Topology

By referring to Figure 5.3 above, we can see that there exists key details, denoted by the large red arrows, that are critical to building a coherent topology. Without these key details, valuable information will be omitted. These details are as follows:

- **Total Number of Inputs**: Denoted by N, this shows the total number of inputs that exist within the network. This is critical for understanding breakdown of figures across the entire topology.

- **Total Number of Inputs per Switch:** This showcases the number of inputs that each switch takes from the input N, and how the inputs are divided up between switches.
- **Connections between Switches**: As the network must route inputs across switches, it is paramount that it is detailed where these inputs are going at every stage of the network. In a simplified topology, like in the above Figure 5.3, we can represent this as a one-to-many mapping where each switch in a stage is connected to every switch in the following stage.
- **Switch Dimensions**: This details the dimensions of the switch, detailing the total number of crosspoints contained within that switch. The dimensions are different depending on various factors as noted previously, such as the current stage.
- **Switch Number**: Denotes the current switch number in the observed stage.
- **Total Number of Outputs per Switch**: Details the number of outputs per switch in the stage k. This is identical to that of the number of inputs in stage 1.
- **Total Number of Outputs:** Shows the total number of outputs, denoted by N. This is identical to the number of inputs, and is equally as important.
- **Stage Number**: Denotes the position of the current stage.

With the formulae for each specific stage outlined, as well as a clear understanding of the construction of a Clos topology, we have all the required information to begin the implementation.

5.3 Choosing our Language and Framework

With the related mathematical formulae gathered and a strong understanding of the many parts of the network topology, the next phase is to build the application. There are a myriad of programming languages we can choose to use, each with their own strengths and flaws. When deciding the best language to use, I decided to opt with Java, for a number of reasons, both objective and personal.

Java

Java is a high level, class-based, object-oriented programming (OOP) language designed by James Gosling, and developed by Oracle corporation in 1995. It is designed as a general-purpose language with the ability to *write once, run anywhere*, provided the machine supports Java. It requires very few dependencies, and remains one of the most popular programming languages today, almost 30 years later.

Within the industry, Java is a popular language for both scientific and engineering purposes. As our application will rely on heavy computations and recursion, it might seem like an odd choice over a language such as Python, and its vast array of libraries to help with such computations or languages with faster execution times, such as FORTRAN. However, Java possesses many traits that aid in the execution of code and managing its performance[23], such as:

- **Just-in-time (JiT) compilation:** JiT compilation refers to the ability to directly convert Java bytecodes into native machine code at runtime. A major benefit of this is tailoring performance optimisations to the execution environment.

- **Garbage Collection:** Java has built-in and automatic memory management that allows the simplifying of memory management. As the application will undergo a large amount of complex computations, this is a must.
- **Adaptive Optimising:** A method of dynamic recompilation of many parts of a program, it allows the optimisation of the application based on the conditions it faces, allowing it tailor its performance based on what it encounters.
- **Hardware Acceleration:** Many graphics engines, like Swing or JavaFX, support utilising hardware such as a computer's GPU to enhance performance in specific instances. As I plan to utilise a graphics engine for this application, it will prove extremely useful if required.
- **Tuning and Profiling:** Java has support for a large variety of tools that allows developers to tune their application to ensure the best performance possible. Such tools are Java Flight Recorder and Java Mission Control, and will prove useful should we encounter any performance issues during development.

In addition to its strengths, another reason for choosing Java was my experience in the language. Having developed applications in Java for almost 3 years, I have gained a substantial amount of experience with the language that I would be able to utilise in this project. My familiarity with Java, alongside my knowledge of various algorithms and data structures, will allow me to use the full potential of the language within my ability, and be more efficient during development.

However, to correctly represent the topology, I would need to utilise a graphics engine within Java to help with constructing the application and drawing the topology. There are many options to choose from, however I chose JavaFX for similar reasons to choosing Java.

JavaFX

JavaFX is a software platform built on and developed for the Java framework. It allows the creation and delivery of rich client applications that feature graphical user interfaces (GUIs), and provides many community-built libraries, tools, frameworks and functions that add to the ease of use in the creation of a GUI-focused application, such as a media player, a chart constructor, game engines and more[24].

There are many features of JavaFX that have made it a good choice compared to other graphics engines in Java, such as Swing. One large feature is its support with JavaFX SceneBuilder, a visual layout tool designed to quickly create user interfaces with a drag-and-drop modality. It significantly cuts down on the development time, as it allows me to perfect the menus and graphics of the application without having to edit much code. The tool produces FXML files, which are not too dissimilar to HTML files used in websites on the Internet. Utilising Controller classes, I can link standard Java code to these FXML files, and produce information regarding the network topology onto rendered diagrams.

In addition, JavaFX has support for many exciting and powerful libraries, alongside supporting features such as:

- **Cross-Platform Support:** As JavaFX uses Java as a basis, it means that it can run on any device that supports Java. Additionally, no extra dependencies are required for JavaFX applications.
- **Web Integration:** JavaFX supports integration with many web technologies, most importantly HTML, CSS and JavaScript, meaning I can deploy the application onto the web, if I wanted to do so.

- **Multimedia Support:** Additionally, JavaFX supports various file types for audio, images, and more, allowing a lot of flexibility with what can be accomplished.
- **Scene Graph Architectures:** As JavaFX uses a scene-graph architecture, it makes the management of user interfaces much easier, and enables more flexible event handling and transformations, among others.
- **Continued Support:** Since the first release of JavaFX, it has enjoyed numerous official and community made updates, with documentation available that covers the intricacies of the tool in-depth.
- **Swing Integration:** While I have chosen JavaFX over Swing, the former contains support for the latter, meaning that I can, in essence, use a combination of both graphics engines in order to achieve the best results.

With our chosen language, graphics engine and all the material required, we can move forward with the construction of the application.

5.4 Code Implementation

When building an application, it is important to ensure that we correctly analyse the scope of it. That is, understanding exactly who this application is for, how the user can interact with it, how it manages data, its performance, and more. Failing to correctly answer these questions can lead to serious development issues within the pipeline, and cause unplanned problems to be dealt with.

To accurately express the timeline of events during the development of the application, I have divided the parts of the development process into the following sections, each detailing what was done, its impact on the overall application, and its influence on the succeeding sections:

Implementation of Formulae

The most logical first step was to correctly implement the formulae for the Clos and Slepian networks, test its performance, understand its resource requirements and make changes to it as necessary. Using the code provided to me by Mr. Wójcik, I began by translating his code from C to Java, and comparing results to ensure it functioned identically. After successful translation, I recorded execution times of far less than 1 second, with identical results produced. For a full analysis of results obtained from testing, see Chapter 6: Results.

Initially, the functions were spread across 3 separate files. After translation, I opted to include every function in a single class, to not only keep them in a single location, but to also cut down on duplicated code, as this was present within Mr. Wójcik's code. The code was modified slightly during this process, as the original code requested an input from the user before executing the method and producing the results. This was removed, and in its place, a generic variable declaration was created. As we shall see during the prototyping and testing phases, this resulted in a significant speed up.

With the Clos formulae in place and working as intended, the next step was to implement the Slepian formulae. As a Slepian network was a variant of the Clos network, I was able to use my translated code as a basis for the Slepian formulae, and stored a copy in a new class for future work. Previously, in Section 5.1, we discussed 2 separate formulae that could be used to calculate

the crosspoints found in a Slepian network. At the time, the 2nd formula was unknown to me, and thus I implemented the first formula. Thanks to its innate nature of being similar to the Clos implementation, I was able to engineer the code with minimal refactoring, but an issue presented early on was execution time performance regarding networks greater than 3 stages. The 5-stage implementation ran noticeably slower than its Clos counterpart, and the 7-stage implementation ran even slower, with execution times upwards of 60 seconds for larger values of N. While some attempts to improve performance were made, none had the impact that I felt was required, and thus this issue was noted for future investigation.

Prototyping

Initially, prototyping began with a paper prototype of the application, take the form of a flowchart. This flowchart detailed how the application operated, using different "states" to show the different parts of the application (I.E. one state would be the "main menu", another would be a "set-up page", and so on). A diagram showcasing one of the first created prototypes can be seen in Figure 5.4.

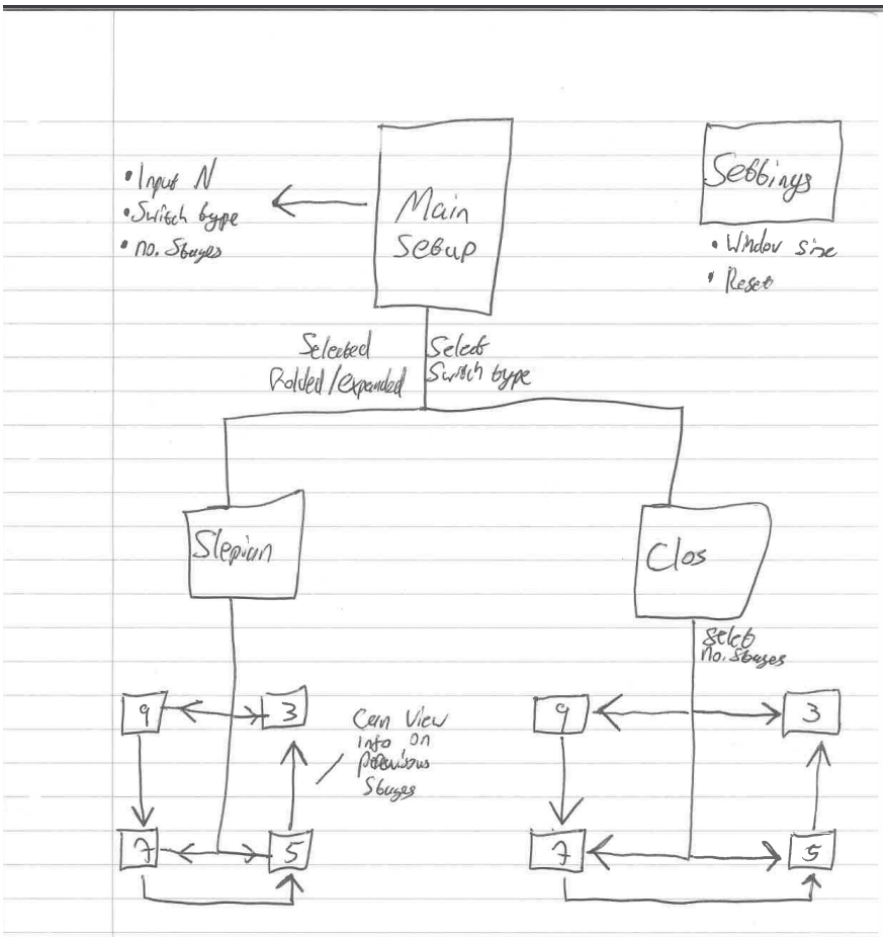


Figure 5.4: Early Paper Prototype for the Network Application

The user would begin from the "Main Setup", where they would be prompted to select the network type they would like to construct. From here, they would either move to the Clos setup "state" or the Slepian setup "state", where they would enter the information regarding the network, such as the number of stages, the input, and any other relevant information. Succeeding this, they would be able to view the constructed diagram for their network. Additionally, a settings tab would be present at every stage of the application, with some options that the user may want to make use of.

This structure for the application, while it went through many renditions, remained largely the same, and a more thorough version, containing extra "states", is present in the final build. With this structure in place, I went about creating a functional prototype to ensure I had the capacity to create the application I envisioned.

When creating the application prototype, I focused primarily on the Clos network. Two of the key reasons for this was using Mr. Wójcik's code as a basis ensured I could focus more on the overarching application, and to spend more time on the implementation of the Slepian network.

Utilising SceneBuilder, I began constructing a simple prototype that would produce the diagram upon execution, as featured in in Figure 5.5. below. Labels, a feature of JavaFX that displays text on the scene shown, was used to set the information required. The Controller class would take the input from the "clos_methods" class prior to displaying the information. When this information was ready, the application would then load the diagram for viewing by the user.

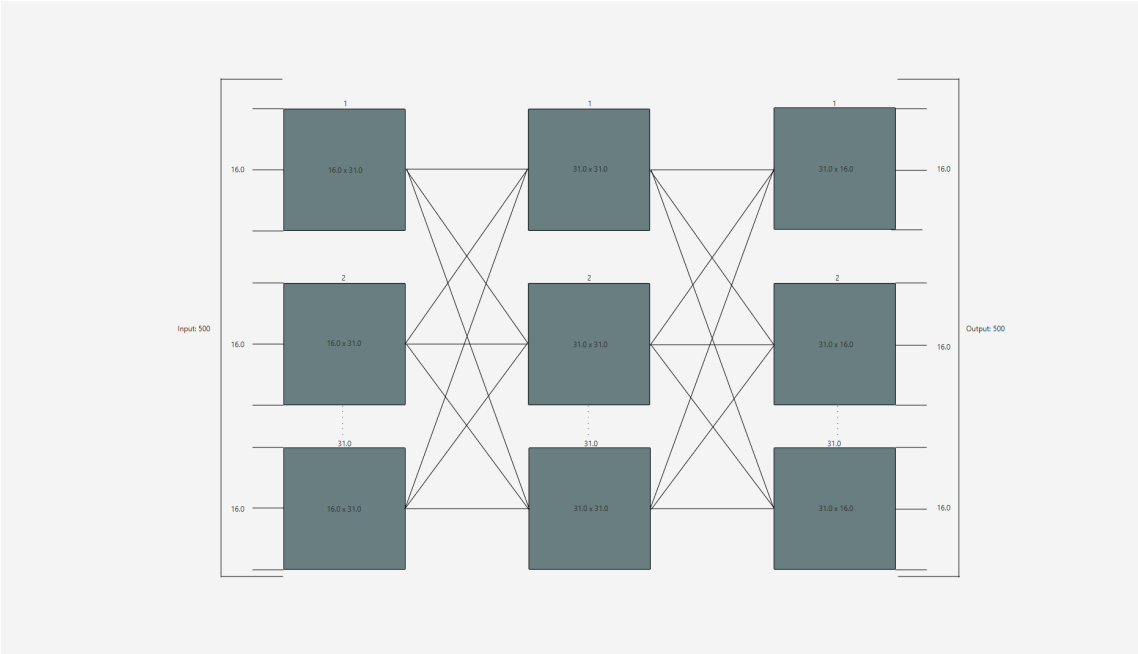


Figure 5.5: Overview of the first functional prototype of the application

In this build, the information for the diagram must be inputted manually into the IDE prior to execution. Additionally, it was limited to only a 3-stage Clos network, however it accurately showcased the application in action. Building this prototype was a fruitful endeavour; it gave me more of an understanding of JavaFX, and what was possible with the engine. When developing the final build, I opted to start from scratch as opposed to building off this prototype. However, I used it as a reference point, and the diagram seen in the final build is an incremental evolution of this prototype.

Not all prototypes are perfect, and I encountered many issues with this prototype in particular. One of the largest roadblocks was ensuring the Labels were set up correctly, so that the information can be passed to them. Simply, it involved defining the Labels within JavaFX, ensuring the controller is linked to the scene, defining and instantiating the Labels within Java via the controller class, and setting the information as a string. As I learned JavaFX, this took a lot of debugging to ensure that the information was correctly recorded and represented on the Labels. However, this problem eased with time and experience.

Overall, while this application went through a number of iterative prototypes, I believe the two showcased above represent significant milestones of the development of this application. Understanding the process of how we arrived at the final build is extremely important, and gives a better

appreciation of not only the final build, but the lessons learned and knowledge gained.

The code for this prototype can be accessed here.

Architecture

The application follows a Model-View-Controller design pattern, as it perfectly suits the needs of what the application requires. This design pattern has been in use since its inception in the 1970s, and is the basis of a wide array of applications created and used today. The pattern consists of 3 distinct parts, that all work in harmony to bring the application to life, being the **Model**, which controls how the information in the application is handled, the **View**, which presents that information to the user. and the **Controller**, which serves to link the former parts together[25].

A diagram of the architecture that describes this pattern can be seen in Figure 5.6 below, with each part coloured differently to represent the role it has in the application. Parts in red represent the FXML files that serve to act as the "View". Models are represented in yellow, and Controllers are represented in green. Additionally, blue represents the main methods that start the application. The View class, represented in orange, contains enumerated types for use by the ViewHandler class.

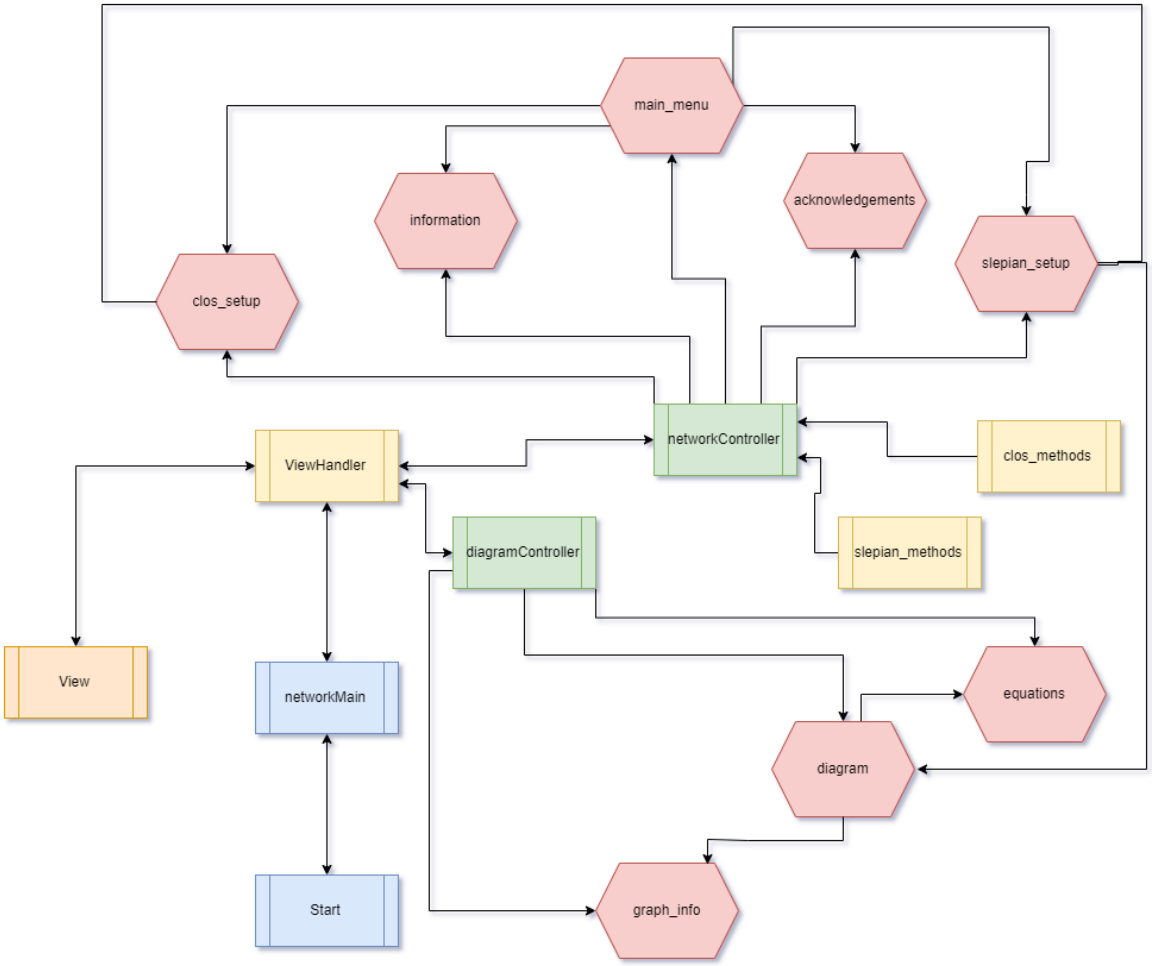


Figure 5.6: Architecture of the Application

Viewing the diagram, you may see that every part is connected to one of the two Controller classes that exist. You may also note that the FXML files and the Model classes are not connected to each other directly, but are instead each connected to the Controller classes which help to process the inputs from one another. Simply, the Controller acts as a "middle man" for the View and the

Model classes, and in the event that one needs to pass information to the other, the information is sent through the Controller who then processes the information. The Model and the View classes never directly interact with each other at any stage of execution.

There is a total of 8 FXML files; 5 of which are connected to the networkController class, with the final 3 connected to the diagramController class. For switching views, they both accept input from the ViewHandler class, who also accepts input from the Controllers to provide the correct file names for view switching. The classes for handling the Clos and Slepian methods are connected to networkController. Additionally, both controllers are connected to one another, so that they can exchange information, specifically information regarding the created networks.

Dependencies

The dependencies of this application consist of a number of built-in libraries supported by Java and JavaFX, meaning no extra libraries were required to be installed or managed. It also allows anyone who wishes to extend this code to do so without any additional work or setup. A table containing information on the libraries used, and a note on their use, can be seen below in Table 5.2.

LIBRARY	USE
java.io.*	Handling input/output and streams
java.util.*	Contains data structures and time measurement methods
javafx.application.*	Contains classes and interfaces for JavaFX classes, and is compulsory
javafx.animation.*	Helps for animation creation in JavaFX applications
javafx.fxml.*	Provides FXML for Java, including loading, parsing and instantiating these files
javafx.scene.*	Contains elements for representing and rendering visual elements of a JavaFX application
javafx.stage.*	Contains classes for managing JavaFX windows, such as creation and configuration
javafx.util.*	Contains utility classes for JavaFX applications for event handling, internationalisation and more

Table 5.2: Overview of libraries used

Classes

In adherence with various software engineering principles, largely influenced by contemporary software development practises, various classes have been created for the purposes of handling a specific function or related functions, and those functions only. This allowed for easier debugging both in static and non-static instances, and ensured that bugs, should they occur, can be easily tracked to their origin. This methodology was adopted largely due to its proven track record of

excellence within the industry[26].

There are 3 categories of classes that exist within the application, based on their role and inspired by the MVC pattern we observed earlier. These include:

- **Method** classes, which contain functions related directly to computations required by the application,
- **Controller** classes, whose role is to interact directly or indirectly with FXML files and apply Java code, either as an official JavaFX Controller class or otherwise, and
- **Main** classes that control the initial execution of the application,.

This unofficial framework served as a great aid for me as I chose where best to implement new features to the code, and allowed the code to remain consistent and focused. In total, **there are 8 classes that are spread across these groups.** They each serve a particular purpose that ensures the smooth running of the application, both on the front end and the back end. An overview of how these classes function, and how they communicate with each other, can be found in the Architecture Section. These classes, based on their category, are as follows:

- **Method**
  - **clos_methods** - This class handles the calculation of the Clos networks. It consists of 3 methods - one for calculating the optimum stage count based on the input, and 2 methods that work alongside each other recursively in order to calculate the main details of the Clos network. Out of these methods, only the main Clos method is public and accessed directly by other classes, adhering to method encapsulation principles. This code implements the previously discussed formulae from Chapter 5.
  - **slepian_methods** - Largely similar to the Clos methods class described above, it contains methods that handles the calculation of the Slepian network. 6 methods are contained within this class, 3 of which are similar to the aforementioned Clos methods. The remaining methods handle calculations for finding succeeding powers based on a given number, finding if a number is a power of the supplied base, and calculating the lowest crosspoints possible from a given input, separate from the main method. This code implements the previously discussed formulae from Chapter 5.
- **Controller**
  - **networkController** - networkController is a JavaFX Controller class that directly interacts with some of the FXML files, as described in the Architecture Section diagram.. It handles the interactive elements of the FXML pages such as buttons, TextFields, and more. Furthermore, it is in this class that the Clos and Slepian methods are called and their execution timed, before the information is displayed on the diagram.
  - **diagramController** - A fellow JavaFX Controller class, diagramController handles the interactive elements of the diagram itself and its associated pages. It shares a lot of the same functionality with networkController, in addition to setting up the diagram for viewing. Extra functions are contained to handle recursive networks, if present.
  - **View** - View contains enumerated types relating to the different pages the user can visit within the application, along with their filename as a string. The class contains a single method, to get the file name of the associated enumerated types. The main use is with the ViewHandler class, as separating these was easier for debugging and extension purposes.
  - **ViewHandler** - This class handles swapping between the above Views while the application executes. Additionally, it handles scene transitions, through a simple fade in, fade out animation.

- **Main**
  - **networkMain** - networkMain serves as the point of execution for the application. It declares the stage to be used, and sets parameters such as the window size and the icon used.
  - **Start** - Issues were encountered when packaging the application whereby the application would fail to start. The Start class solved this issue by nominating this as the main method in IntelliJ IDEA. Its purpose is simply to call Main above, where it would execute as normal.

Layout

During the initial stages of prototyping, it became evident that the user interface (UI) design for this application demanded simplicity and a strong emphasis on accessibility. Furthermore, it was imperative that the interface remain intuitive and easily navigable. However, as I embarked on this endeavour, I encountered a significant challenge owing to my limited experience in UI design. Over time, this obstacle gradually subsided, particularly following the implementation of the Main Menu—a pivotal milestone that laid the foundation for all subsequent pages.

Constructed using SceneBuilder, the Main Menu boasts a minimalist layout, featuring four prominently displayed buttons for user interaction alongside a large heading welcoming the user. The menu offers essential options such as switch type configuration, comprehensive switch information and user-friendly tutorial, and an acknowledgements section. This central hub not only establishes a standardized design paradigm but also sets the tone for the application's visual identity. Subsequently, the Set-up, Acknowledgement, and Information views adhere closely to this established design language, ensuring a cohesive and intuitive user experience.

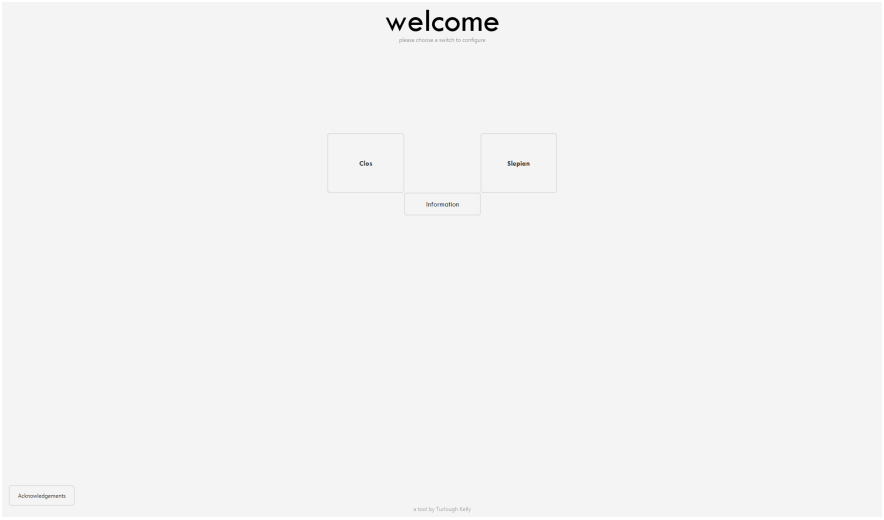


Figure 5.7: Main Menu of the application

Strategically, the placement of vital options in the centre column of the screen facilitates easy discoverability and interaction. Throughout the application, the UI makes use of a subdued colour palette, primarily comprising neutral hues of white and grey, imbuing the application with a sleek and contemporary aesthetic. The judicious use of blue accents, notably within the diagram depicting switch configurations, serves to delineate key components and enhance visual clarity. Additionally, red is used during the tutorial that serves to teach the user on reading the diagram correctly. This deliberate design approach not only prioritizes user accessibility but also elevates the overall user experience, fostering a sense of familiarity and ease of use, ensuring bright colours are used to denote areas of importance.

Testing

Two types of testing were conducted on the application - one that aimed to test the Clos and Slepian methods that were implemented, and one to test the overall functioning of the application. The former was carried out via unit tests through JUnit, while the latter was tested manually.

The methods were slightly modified in order to return an array of double values, with either 5 or 6 elements depending on the method tested. These arrays were then asserted through JUnit to confirm if the method passed the test or not. A minor aspect of test-driven development (TDD) was applied, mainly through the Slepian methods. In total, there was approximately 50 unit tests per network type, testing values from the minimum accepted value, being 3, all the way to approx. 5 million. The Clos and Slepian methods both used the same inputs for their tests. Separate unit tests were conducted for the supporting methods of each network, and totalled to 101 tests in total.

For testing the application, this was conducted manually, through executing the application and interacting with it directly. While not completely foolproof, I felt it was adequate given the limited control users have over the application as a whole when it came to options of interaction. The main cause for concern was inputting data to create the Clos or Slepian networks, however this was mitigated through the TextFormatter implemented that blocks the input of non-numerical characters.

Additionally, as mentioned above in the Layout section which details the application interface design, I observed as participants interacted with my application, and invited them to attempt to cause any sort of failure of the application. The feedback collected through this process was then applied to fixing any issues encountered, to which mainly came in the form of visual bugs that did not affect functionality of the application in any way. This type of testing continued until the application was at an error-free and stable build.

Performance Enhancements

As the methods used for calculating the Clos and Slepian methods initially relied heavily on recursion, I experimented in many different avenues to speed up the execution of the program. Ensuring a fast execution time was critical, as a slow execution of the network details can lead to a slowdown of the application as a whole, resulting in a potential crash. Avoiding this in the final build was my top priority, and during testing, this issue was more prevalent with the Slepian network, with the Clos network consistently executing quickly for manual stage specifications. All results discussed in this section can be viewed in the Results chapter.

During testing, the execution time for Clos networks remained consistent, increasing with respect to the input N. As mentioned previously, the code was expanded to include methods to optimise the stage count. However, the execution time will only become somewhat of an issue for very large values of N. Despite, I felt there was no need for performance enhancements at this time, as the code performed well from early implementations. The data concerning execution times for Clos networks can be found in the next chapter.

Concerning the Slepian network, execution times for this network were extremely slow at first. The first draft of code was structured quite similarly to that of the Clos network, given their innate similarities, and thus involved recursion during the calculations. However, it became apparent quite quickly that this would mean quite a severe slowdown, as the number of stages present in the Slepian network (and thus the depth of the recursion) are much larger than that of the Clos network. This would result in large slowdowns for networks of 13 stages or more, and computations would take minutes at a time. This problem was noticed quite early on, and thus could be rectified in good time.

One avenue that I explored to combat this was the implementation of parallelisation into the Slepian methods. This would mean splitting the workload of the application across multiple processors and threads in the system in an attempt to result in a speedier execution time. Using Java's Concurrent library, an Executor object would be created and map itself to any available processors that the machine had available, before loading the Slepian workload into those processors. Finally, after all the calculations were complete, the Executor would merge the processor's jobs and produce the final output. While this saw an decrease in execution time, it was noted to use up quite a lot of the computer's available resources. This meant that only moderately powerful computers would be capable of running the application in good time, and was another issue that I wanted to avoid.

After some time had passed and experimenting conducted, the second formula for calculating Slepian networks, as detailed previously, was implemented. This implementation completely removed any recursion from the calculations, and was best suited for the degree of scaling that the Slepian network may undergo in relation to stage count with respect to input. This significantly reduced the number of calculations required in order to calculate the number of crosspoints, to potentially 1 (if the network was optimised), to at most $N/2$, where N is the number of inputs in the network. This is a drastic change to the previous methods, where the maximum number of calculations would be $((k-1)/2) * (N/2)$, where k is the number of stages, and N is once again the input. This reduction saw execution times on par to, and in many cases better, than that of the Clos network, in addition to a reduced use of the computer's resources. Once again, these results can be viewed in the Results chapter.

Overall, enhancing the performance saw me experiment with many different options in order to find execution times that would be deemed acceptable for the application, including parallelisation of the Slepian network. Overall, the execution times that currently exist in the application are ones that I am very happy with, as it ensures a smooth and snappy user experience, and not one plagued by crashes and time-out errors.

Error Handling

Given the layout of the application, there are 2 main areas where errors can occur: the setup pages, as well as the diagram itself. Outside of these two areas, input from the user is controlled strictly through the use of on-screen buttons. No other input is possible, and throughout testing no errors have occurred through these buttons.

Beginning with the setup pages, errors can occur if an input that cannot be handled by the application is processed. This largely includes symbols and characters, and processing these inputs will lead to exceptions thrown and a failure to load the succeeding page. To combat this, the TextField instances, where users can directly input to define the number of inputs as well as the switch size (in the case of the Slepian switch), were combined with UnaryOperators, built directly from the java.util library, and TextFormatter, from the javafx.scene library. These created a "filter", which will reject changes that does not match the specified parameters. Simply, it will reject any input that is not a number. This has helped to forgo the need of handling inputs directly, as the only characters that are permitted to be entered are numerical. In the event that a null input is submitted, or that the number entered is too low (I.E. 0, 1 or 2), a Label will print a message in red denoting that the input is not valid. **The exact code can be found in the "networkController" class on GitLab.**

For the diagram, the errors that can be encountered originate from the built Clos and Slepian methods, and not from the diagram itself (or its associated code). There were some cases where, mainly due to the input, that the methods would return a 3-stage network with 0 for all associated variables. This was due to difficulties in the formulae with the given inputs (I.E. the total number of inputs were not easily divisible by the switch size). This error only occurred with unoptimized networks. In an effort to correct this, a simple change was made: in the event that the unoptimized

network was not possible and gave the errors above, it would simply optimise the network based on the given input and switch size (if applicable). This single change eliminated the problem outright, as the optimisation algorithm would ensure that if the given input was not possible, the next best network would be found. This would ensure that, in any event, a network was always supplied, either being the one specified or the next best result. **This exact code can be found in either the "clos-methods" or the "slepian-methods" classes on GitLab.**

Documentation

Documentation regarding the application is mostly self-contained within the application itself. Accessible from the main menu is an "Information" page that details notes on the application, such as what it is, what it hopes to achieve, and how to use it, alongside some introductory information on what Clos and Slepian networks are.

When viewing a constructed diagram of the network, a page titled "Equations Used" becomes accessible. It details largely the same information as described in Section 5.1, showcasing what formulae are used, as well as examples that showcase the formulae in action. It gives the user an insight as to how the application functions, but it also acts as a resource for users to take the formulae for application outside of the program.

Contained on the GitLab page is a README file that describe how the application is intended to run, and instructions on running it correctly. It also outlines where to find the above information within the application.

Version Control

Version Control was handled largely through GitHub. As I had used IntelliJ IDEA as my preferred IDE, it supported direct GitHub integration that allowed me to continuously commit changes as they were made, ensuring that a backup of my code was in GitHub at all times. Additionally, it made it easy to work on the application across multiple systems when the situation demanded it. In general, I kept to at least one commitment and push per hour that I worked on the project, although this wasn't always the case.

When the application's final build was complete, a copy of the code was pushed to GitLab, to allow for sharing among members of UCD's faculty. A link to this repository can be found here.

Deployment

The application is designed for local deployment, to be run as an executable .jar file packaged with Maven via IntelliJ IDEA. Not only does this mean it can be run on any machine that supports Java, it also does not require an internet connection in order to function. Furthermore, it allows for much easier distribution, as the .jar file is quite small in size. While this means that pushing updates, should it be necessary, can pose an issue as people will need re-download the application, I do not see this as a major issue. Given the completeness of the application as this moment, future updates (if any), would be minor, only focusing on the aesthetics of the application, and not its core functionality. This means that no matter the version in possession, the core functionality will remain unchanged.

Chapter 6: **Results**

During the development of the application, a number of tests were conducted for 2 primary reasons: to measure the overall execution time of the program to ensure that it is acceptable as the input N increases, but also to test that, when optimised, the number of stages scales correctly to account for the input N. Relating to the Slepian network, I also detailed the changes in execution time as more methods were experimented with. This chapter aims to showcase these results and more, showcasing the speed of the program, but also providing context to the user for what switch size should be used, in the event that they would like to construct their own, as well as reinforce the claims made throughout the paper in relation to network size and growth as the input for a network increases.

Clos Networks

The graph detailed in Figure 6.1 details the execution time relevant to the input for Clos networks.

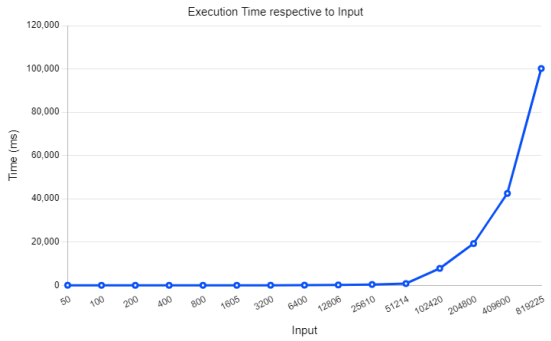


Figure 6.1: Execution Time respective to Input when Optimising a Network automatically

Initially, we can see that the first input of 50 begins with a relatively quick time of 50 milliseconds, however it should be noted that the next input of 100 produced a time of 32 milliseconds. However, succeeding this the time decreases quite a lot to a mere 1ms, before steadily rising again as N reaches quite large numbers. The risk of the program getting slow is only an issue at very large values for N. For smaller values of N, the program will run to a very satisfactory standard.

The reason for this increase of execution time is the extra time required to calculate the optimal stage count for that given input. In essence, the Clos network is calculating up to 2 additional crosspoints (for $k-2$-stages and $k+2$-stages) for comparison. If this were to be omitted, then the execution times would decrease. By selecting the stage manually, we can drastically decrease the execution time, as seen in in Figure 6.2.

We can note that optimising the network manually and optimising the network automatically follows an almost identical trend as input increases. However, we can also see that selecting the stage count manually drastically reduces our execution time. In the case of $N = 819225$, the automatic optimisation time is 100166ms, as opposed to 12107ms for manually optimising. This is a speed increase of nearly 880%, which is quite substantial. However, I believe having this option in place is critical for those without any prior experience in computer networking and with these types of networks.

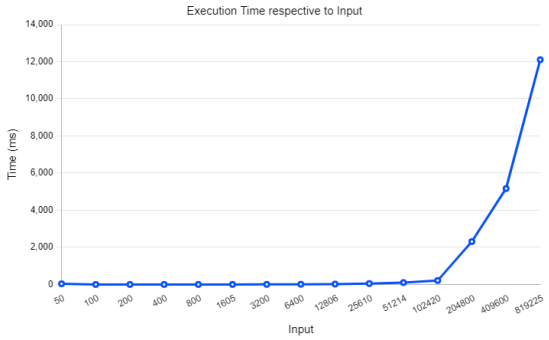


Figure 6.2: Execution Time respective to Input when Optimising a Network manually

As the input for the Clos network increases, so too will the number of stages present in the network. While it may never reach the number of stages that the Slepian network may possess, the Clos network will still possess an acceptable number of stages. The following graph showcases the number of stages with respect to the input:

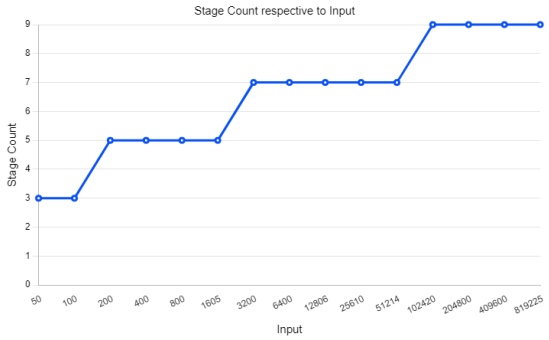


Figure 6.3: Execution Time respective to Input when Optimising a Network manually

We can note that the networks reach a size of 5-stages early on, with an input of 200. The next change in stages does not come until $N = 6400$, whereby it changes to a 7-stage. It is possible that for much larger inputs, greater than 819225, that a Clos network may reach a size of 11 stages and possibly even 13 stages. However, based on this trend it would take an input of millions in order to achieve such a feat.

Overall, we can note that the Clos network performs as expected, however it does fall below expectations when optimising the network for much larger inputs of N. This is something that can be investigated in the future, as I believe there is a way to more effectively calculate the correct number of stages without having to compare multiple instances of a network. This aside, the Clos network provides a quick execution time that may be possible to optimise further.

Slepian Networks

As noted in in Chapter 5, the Slepian formulae went through a number of iterations before finalising the formula used. As a result, the execution times have varied quite substantially. A graph detailing all these results can be seen below. In all instances, the value of r was set to 2, and the most optimum stage count for each input was manually calculated prior to execution.

We can immediately note that the execution times for the Original Implementation and the Parallel Implementation are exceptionally high, reaching over 900000 seconds in each case, or about 10

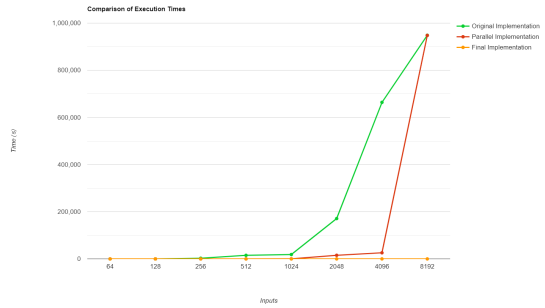


Figure 6.4: Execution Time respective to Input when Optimising a Network manually

days of execution time. A trait noted previously in in Chapter 5 is that Slepian networks will reach much larger stage counts that their Clos counterpart. In total, $N = 8192$ produced a stage count of 25, which means that these networks would have to be calculated through 11 nested recursive calls. Despite being paralellised, the Parallel Implementation of the network still preformed extremely slowly. These tests were conducted using a computer with an Intel Core i7-10750H processor clocked at 2.60GHz, using a PC611 NVMe SK Hynix 1TB solid state drive (SSD), along with 16GB of DDR4 SODIMM RAM, using the Windows 11 OS. The main reason for such a slow implementation is mainly due to the immense amount of recursive calls that the program needed to make.

When testing the new implementation, which is used in the final build, the results are a stark contrast to the results observed previously. The new implementation, unlike the original methods, does not use recursion at any stage, with the crosspoint calculation handled by a single line of code. This removal of recursion speeds things up magnificently, with every single execution time totalling 0 seconds at its slowest, compared to 10 days with the other methods. While a speedup of execution time was required, this level of improvement was completely unprecedented. It will be worth investigating if the Clos network can be similarly improved to remove recursive calls, and reach execution times on par with the final Slepian implementation.

To further illustrate the capability of the Slepian network implementation, Figure 6.5 showcases the execution times up to an input of 819200. These times include automatic optimisations with a r value of 2.

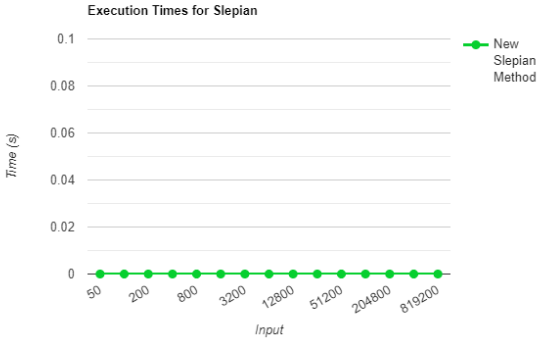


Figure 6.5: Execution Time respective to Input for Slepian Networks

The execution time remained consistent overall, staying at 0 seconds throughout. Without the need for recursion, the program performs exceptionally fast, and provides users with an immediate answer when constructing their network. Once again, this is a monumental improvement over the former implementations of the Slepian formula.

Previously, we noted many times that the Slepian network will reach much larger stage counts as the input increases when compared to the Clos network. The following graph aims to showcase this information, so that you may gain an understanding of this.

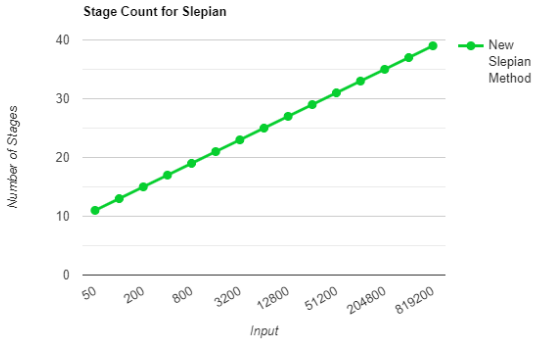


Figure 6.6: Number of Stages in a Slepian network respective to Input

We noted that for an input of 50, a Clos network consisted of only 3 stages. We can see that, for a Slepian network with the same input, that the number of stages present is 11. For this test, the input began at 50 and increased by $2x$ for each iteration, with x equating to the current number of inputs. We can see that, as the input increases linearly, so too does the number of stages, increasing by 2 as the input grows. It should be noted that for many values, this trend may not be strictly linear, as many factors, such as the switch size and input, must be taken into account.

Overall, the analysis of these results serves as a good indication of the application's speed and optimisation, whilst also identifying areas of improvement, particularly regarding the Clos network. It also showcases how the Clos and Slepian networks increase the number of stages as the inputs increase. Additionally the effectiveness of the implemented formulae is demonstrated, and in the case of the Slepian network, the incremental speed increases as more methods were experimented with.

Chapter 7: **Future Work**

The final implementation of the application does not leave a lot to be desired. Comparing it to the original project specification and the objectives I laid out in Chapter 1, I have successfully achieved all but one of these objectives through this implementation. I have also provided the groundwork within the application such that, if I or another person wanted to expand the switches that could be constructed, such as a Moore switch, that it would be relatively straightforward to do so.

The single objective that was outlined in Chapter 1 but not completed was Goal 1.2.2, which can be seen using the link above. The core reason as to why this objective was not implemented was due to the execution time in early tests, which did not meet the standard that was required. As you can see in Chapter 6, relating particularly to the Slepian network, initial execution times were astronomical for larger values of N, and proved to be an issue. At the time, I did not thoroughly experiment with avenues of optimisation, owing to why I did not pursue this goal further. At present, I believe that the current advancements I had made to optimising the execution time within the program will allow this goal to be seen to completion, either by me or by other developers. Should I revisit this project in the future, this will be the first goal I would work towards.

There were some goals that had to be reconsidered throughout the course of development, of which the most notable was the implementation of a Folded Clos network. This sub-type of Clos networks would represent the same information as the implemented network, but would take on a different appearance. In a practical sense, these networks are more cost efficient, yet far less flexible in terms of scalability when compared to its expanded compatriot. These networks also contain less stages, as many of them are "folded" onto one another, producing a more compact network. Ultimately, they would represent the same network, in two different forms (cite folded paper). Thus, I did not see any need to implement the same type of network twice without good reason to, and did not go ahead with it. Perhaps an expansion to this application, alongside entirely new switch types, could see the implementation of these Folded networks.

One final goal that was entertained but not implemented was providing information, based on the currently generated network, on how the inputs would affect the crosspoint count (and possibly stage count) of the application, showcasing a need for scaling should more inputs be required. While an interesting thought, at the beginning I opted to implement the diagram first, and leave any additional goals such as these until after the main application had finished. As time went on, due to academic pressures and setbacks in development, this idea was eventually scrapped. However, one day I believe that this would be good information to provide to users, and could be left to future developers who wish to expand this application's utility.

Observing the results obtained in in Chapter 6, notably regarding the automatic optimisation of Clos networks, may inspire others to better optimise the code used within the application for finding the optimal stage count automatically.

Overall, the future work that could be pursued in relation to this application comes down to quality of life improvements and an expansion to the application's already present functionality. I firmly believe that the application, in its current build, does not strictly require any additional modification at present. However, I have ensured that the code provides avenues for easy expansion, such that should future work be pursued, that it can easily and seamlessly fit into the exist framework of the application without major refactoring.

Chapter 8: Summary and Conclusions

This thesis oversaw the key details and challenges for the development journey of an application tailored for designing optimal switching networks. Initially, we delineated a set of objectives guiding our implementation efforts. Reflecting on these objectives, we accomplished all Core goals outlined in Chapter 1, alongside the attainment of the initial Advanced goal. Regrettably, due to time constraints, the realization of the final Advanced goal was unattainable within the project's scope. However, this limitation has been duly acknowledged in the Future Work section for prospective integration. Nonetheless, the application stands as a testament to its completeness, effectively achieving its intended purpose.

Through our exploration, a niche in diagram-drawing applications was discovered during comparisons with existing solutions. Upon noting the advantages and disadvantages of these solutions,, we saw how the application satisfactorily fulfils the aforementioned niche and addresses the issues of similar software. We concluded that the use cases for each solution varied, and there did not exist one single solution that would fit every scenario. Instead, we ensured our application stood alongside these applications, and not necessarily replaced any one of them.

Significant attention was dedicated to understanding the foundational principles underpinning Clos and Slepian networks. Drawing from a myriad of sources, we successfully collected and analysed the most important concepts relating to the construction of a Clos or Slepian topology. This has allowed this paper to serve as a road map for understanding these networks. To support this, examples were detailed such that this material can be easily understood and applied by a large array of users, including those without a background in computer networking or mathematics.

This thesis described in detail the process that was undertaken to construct this application. Agendas in the forms of Gantt charts described the work completed during two consecutive periods of the academic year, showcasing important areas of learning, experimentation and implementation. From this, a breakdown of the application as a whole was provided, showcasing the architecture that the application follows, and how each part of the application is connected.

Furthermore, we embarked on envisioning future avenues of improvement for the application, predominantly centred on aesthetic enhancements and the expansion to encompass other network switches. Additionally, we presented empirical findings regarding execution time dynamics, revealing discernible patterns as network scale increases, as well as how the networks may grow as the input increases. Noteworthy improvements in execution time, particularly concerning Slepian networks, underscore the evolution from preliminary implementations to the final iteration.

In conclusion, this project successfully completed the outlined objectives and showcased in great detail the journey undertaken to do so. I believe that this project, application and paper both, can contribute to the greater world of networking in its current form. There is much improvement to be had and optimisations to make, however, and it is equally as important to note the work uncompleted or not undertaken throughout this process. Despite this, this project represents more than just the culmination of academic endeavours—it symbolizes a step forward in addressing real-world challenges in networking. By leveraging the insights gained and the tools developed, we have the potential to revolutionise the way switching networks are designed and deployed, and it is my hope that this project contributed to that in some capacity.

Acknowledgements

My supervisor, Professor John Murphy, whose knowledge and guidance was paramount throughout, and was always ready and willing to help flesh ideas out or provide different views at various stages of this project. I would've been lost without him. Thank you, John.

Andrzej Jajszczyk and Robert Wójcik, as their paper, *The Enumeration Method for Selecting Optimum Switching Network Structures* was a major influence for the code written, and who were kind enough to provide me with, and allow the modification of, their code from the original paper.

My friends and family for their constant support and for acting as a wall to bounce ideas off of, even if they had no idea what I was talking about. Thanks guys!

My peers and all staff from the University College Dublin School of Computer Science, who equipped me with the knowledge to tackle such a large undertaking in my undergraduate degree.

Bibliography

1. Clos, C. A Study of Non-Blocking Switching Networks. *The Bell System Technical Journal* **32,** 406–424 (2 1953).
2. Hojabr, R., Modarressi, M., Daneshtalab, M., Yasoubi, A. & Khonsari, A. Customizing Clos Network-on-Chip for Neural Networks. *IEEE Transactions on Computers* **66,** 1865–1877. ISSN: 00189340 (11 Nov. 2017).
3. Sehery, W. & Clancy, C. Flow optimization in data centers with Clos networks in support of cloud applications. *IEEE Transactions on Network and Service Management* **14,** 847–859. ISSN: 19324537 (4 Dec. 2017).
4. Benes, V. E. *On Rearrangeahle Three-Stage Connecting Networks* 1962.
5. Whithing, T. & Hah, T. K. Machine Learning with Clos Networks. http://arxiv.org/abs/1901.06433 (Jan. 2019).
6. Turner, J. S. & Melen, R. Multirate Clos Networks. *IEEE Communications Magazine, Vol 41, No 10,* 38–44 (2003).
7. Pun, K. & Hamadi, M. Dispatching schemes for Clos-network switches. *Computer Networks* **44,** 667–679. ISSN: 13891286 (5 Apr. 2004).
8. Walsh, J. P. & Bayma, T. *Computer Networks and Scientific Work* 1996.
9. Office, U. S. P. & Erwin, E. L. *Patent for the original Telecommunication Switch* 1938.
10. Karimi, A., Aghakhani, K., Manavi, S. E., Zarafshan, F. & Al-Haddad, S. A. *Introduction and analysis of optimal routing algorithm in benes networks* in. **42** (Elsevier B.V., 2014), 313–319.
11. Jajszczyk, A. & Wójcik, R. The Enumeration Method for Selecting Optimum Switching Network Structures. *IEEE COMMUNICATIONS LETTERS* **9.** http://www.kt.agh.edu.pl/ (1 2005).
12. Paap, K. R. & Cooke, N. J. *Chapter 24 Design of Menus*
13. Gao"', B., Hwang, F. K. & Du, D.-Z. *Wide-sense nonblocking for multirate 3-stage Clos networks* 1997.
14. Esmaeel, Z., N., F. M. & Mohammad, T. Optimal Design of Switch Sizes in Strictly Non-Blocking Clos Three-Stage Interconnection Networks.
15. Hwang, F. Control Algorithms for Rearrangeable Clos Networks. *IEEE Transactions on Communications* **31,** 952–954 (1983).
16. Morimura, T., Iwai, K. & Amano, H. Hierarchical multistage interconnection network R-Clos. *Electronics and Communications in Japan, Part III: Fundamental Electronic Science (English translation of Denshi Tsushin Gakkai Ronbunshi)* **89,** 30–39. ISSN: 10420967 (11 Nov. 2006).
17. Slepian, D. Two theorems on a particular crossbar switching network. *Unpublished manuscript* (1952).
18. Chung, F. R. K. An Algebraic Approach to Switching Networks. *Bell Laboratories.*
19. Jajszczyk, A. *Optimal Structures of Benes' Switching Networks* 1979.
20. Nassimi, D. & Sahni, S. A Self-Routing Benes Network and Parallel Permutation Algorithms. *IEEE Transactions on Computers* **C-30,** 332–340 (5 1981).
21. Nikolaidis, D., Groumas, P., Kouloumentas, C. & Avramopoulos, H. Novel Benes Network Routing Algorithm and Hardware Implementation. *Technologies* **10.** ISSN: 22277080 (1 Feb. 2022).

22. Benes, V. E. *Mathematical Theory of Connecting Networks and Television Traffic* (Academic Press, 1965).

23. Labs, C. S. *Algorithmic Performance Comparison Between C, C++, Java and C Programming Languages* 2010. http://www.cherrystonesoftware.com.

24. Pedersen, S. B., Jackson, S. & Switzerland, G. GRAPHICAL USER INTERFACE PROGRAMMING CHALLENGES MOVING BEYOND JAVA SWING AND JavaFX (2019).

25. Gupta, P. *MVC Design Pattern for the multi framework distributed applications using XML, spring and struts framework* 2010.

26. Pargaonkar, S. Cultivating Software Excellence: The Intersection of Code Quality and Dynamic Analysis in Contemporary Software Development within the Field of Software Quality Engineering. *International Journal of Science and Research (IJSR)* **12,** 10–13 (9 Sept. 2023).